



ค่ายโอลิมปิกวิชาการวิชาคอมพิวเตอร์ ค่าย 2
ระหว่างวันที่ 18 เมษายน ถึง 30 เมษายน พ.ศ. 2566

โครงสร้างข้อมูลกราฟ 2 และอัลกอริทึมกราฟ

อ.ลือพล พิพานเมฆาภรณ์

luepol.p@sci.kmutnb.ac.th

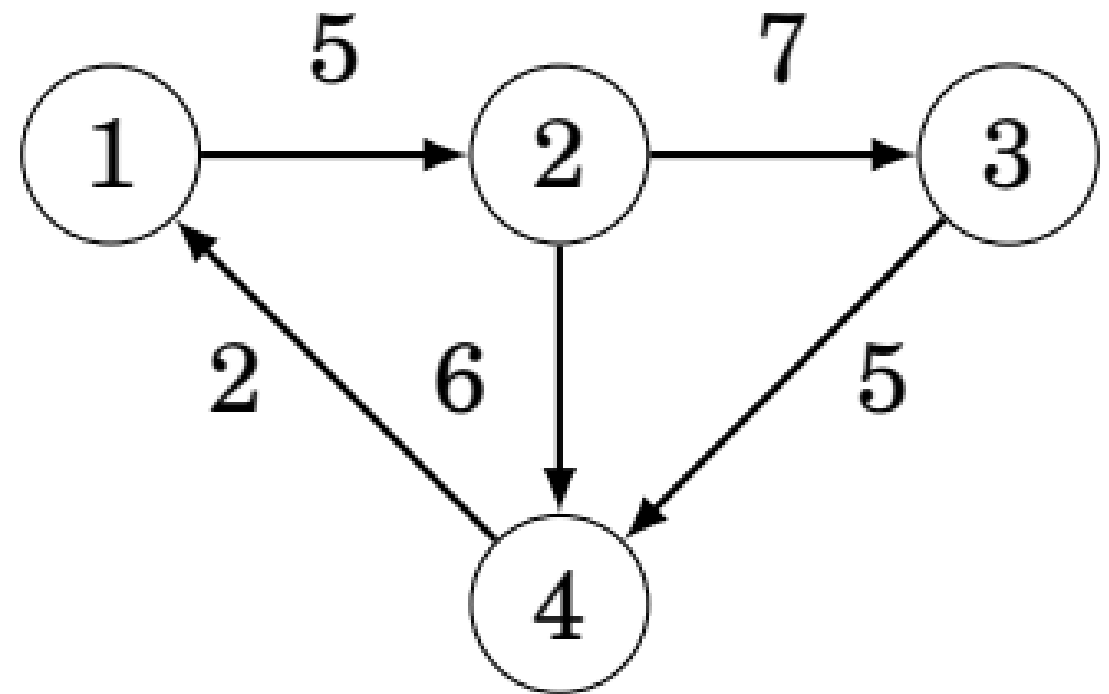
ภาควิชาวิทยาการคอมพิวเตอร์และสารสนเทศ

มหาวิทยาลัยเทคโนโลยีพระจอมเกล้าพระนครเหนือ

- กราฟชนิดถ่วงน้ำหนัก (Weighted graph)
 - เมตริกซ์ประชิด (Adjacency matrix)
 - ลิสต์ประชิด (Adjacency list)
- ปัญหาการหาเส้นทางสั้นที่สุด (Shortest path) ในกราฟ
 - Single-source shortest path problem
 - All-pair shortest path problem
- ปัญหาต้นไม้แผ่ทั่วที่เล็กที่สุด (Minimum spanning tree) ในกราฟ
 - Prim's algorithm
 - Kruskal's algorithm

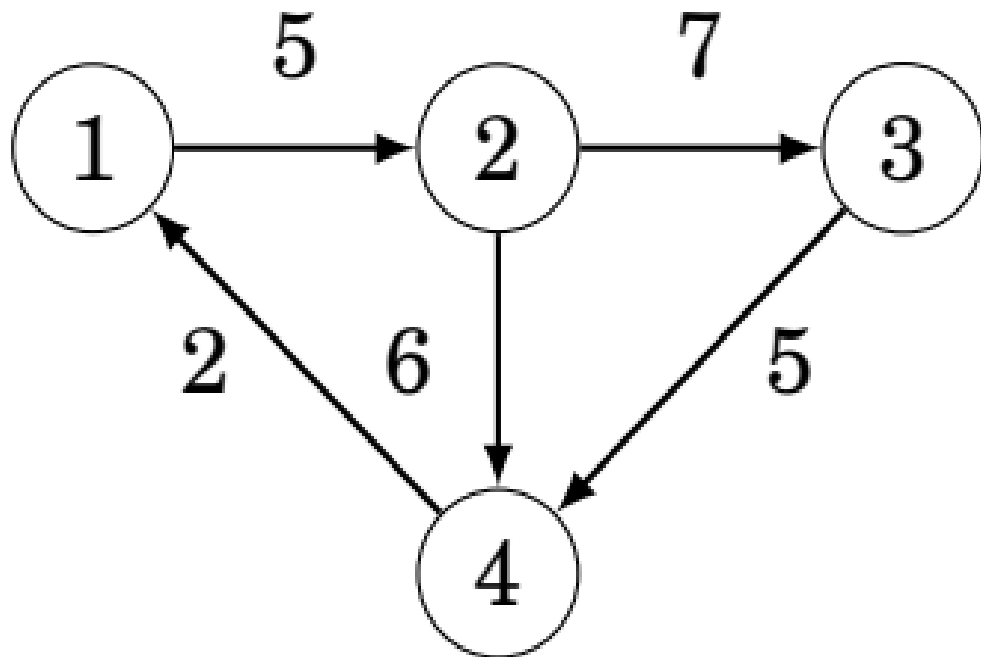
กราฟชนิดถ่วงน้ำหนัก (Weighted graph)

- กราฟชนิดถ่วงน้ำหนัก (weighted graph) เป็นรูปแบบกราฟที่มีการระบุค่าน้ำหนัก (weight) ให้กับเอจในกราฟ
- ค่าน้ำหนักอาจแทนด้วยระยะทาง (distance) คะแนน (score) เวลา (time) หรือเงื่อนไขอื่นขึ้นอยู่กับการใช้งานกราฟ
- การแสดงกราฟชนิดถ่วงน้ำหนักทำได้ 2 แบบ ได้แก่
 - เมตริกซ์ประชิด (adjacency matrix)
 - ลิสต์ประชิด (adjacency list)



เมตริกซ์ประชิด (Adjacency matrix)

```
int adj[N][N];
```



เวอร์เท็กซ์
เริ่มต้น

	1	2	3	4	เวอร์เท็กซ์ ปลายทาง
1	0	5	0	0	
2	0	0	7	6	เวอร์เท็กซ์ เริ่มต้น
3	0	0	0	5	
4	2	0	0	0	ค่าน้ำหนัก

การเข้าถึงสมาชิกในเมตริกซ์ประชิด

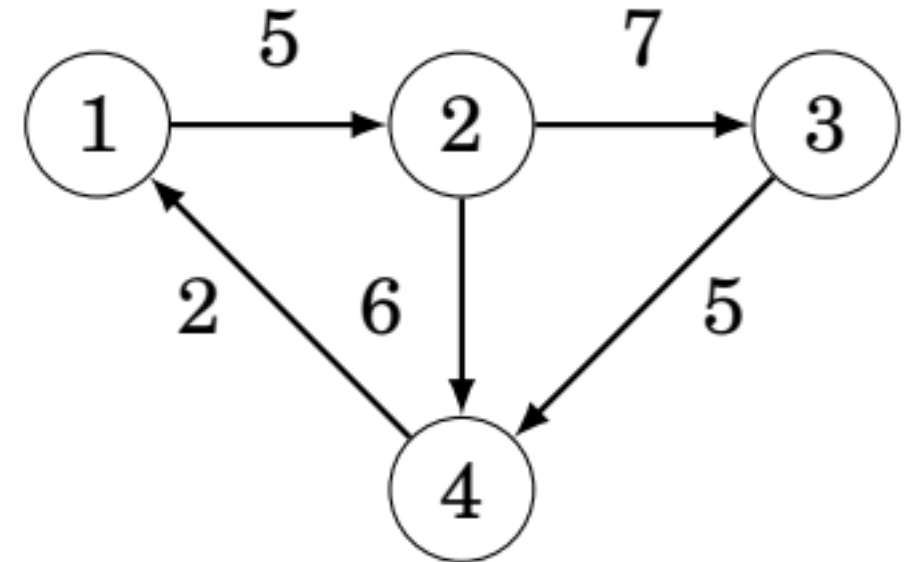
```
#define V 10
```

```
int main()
```

```
{ int adj[V][V];
  memset(adj, 0, sizeof(adj));
```

```
adj[1][2] = 5;    adj[2][3] = 7;
adj[2][4] = 6;    adj[3][4] = 5;
adj[4][1] = 2;
```

```
for(int i=0; i<V; i++)
{ for(int j=0; j<V; j++)
  { if(adj[i][j] != 0)
    printf("(%d, %d, %d)\n", i, j, adj[i][j]);
  }
}
return 0;
}
```



C:\Users\ZenBook\Desktop\Untitled4.exe

```
(1, 2, 5)
(2, 3, 7)
(2, 4, 6)
(3, 4, 5)
(4, 1, 2)
```

ลิสต์ประชิด (Adjacency list)

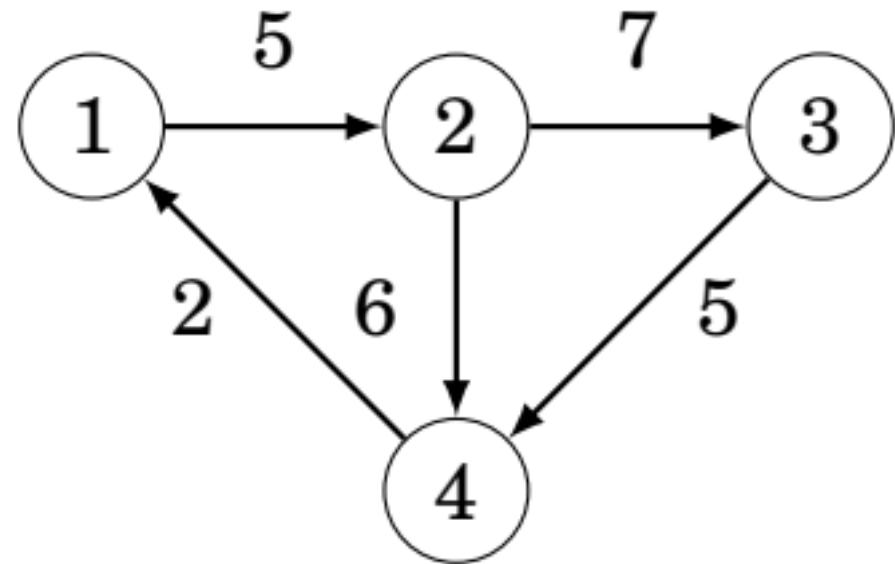
- สร้างอาร์เรย์ของเวกเตอร์ (vector) โดยสมาชิกแต่ละตัวจะแสดงในรูปแบบของคู่ลำดับ `pair<int, int>`
 - `pair.first` แทนเวอร์เท็กซ์ปลายทาง
 - `pair.second` แทนค่าน้ำหนัก (weight) ระหว่างเวอร์เท็กซ์

```
vector<pair<int,int>> adj[N];
```

```
int main()
{
    vector<pair<int, int> > adj[4];

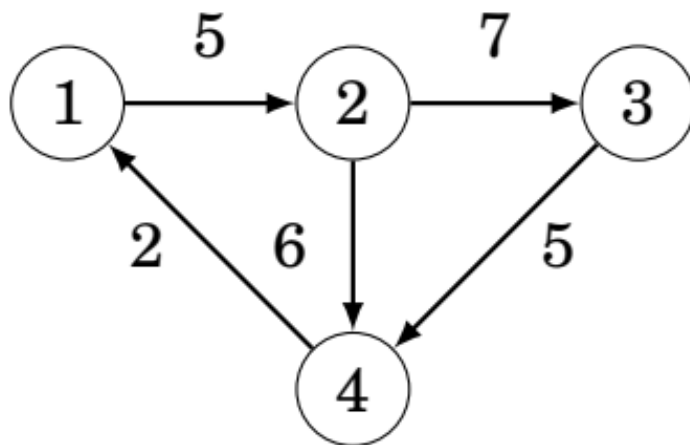
    adj[1].push_back({2, 5});
    adj[2].push_back({3, 7});
    adj[2].push_back({4, 6});
    adj[3].push_back({4, 5});
    adj[4].push_back({1, 2});

    return 0;
}
```



การเข้าถึงสมาชิกในลิสต์ประชิด

```
void printGraph(vector<pair<int, int> > adj[], int V)
{
    int v, w;
    for (int u = 0; u < V; u++) {
        for (auto it = adj[u].begin(); it != adj[u].end(); it++) {
            v = it->first;
            w = it->second;
            cout << "(" << u+1 << ", " << v+1 << ", " << w << ")" << endl;
        }
    }
}
```

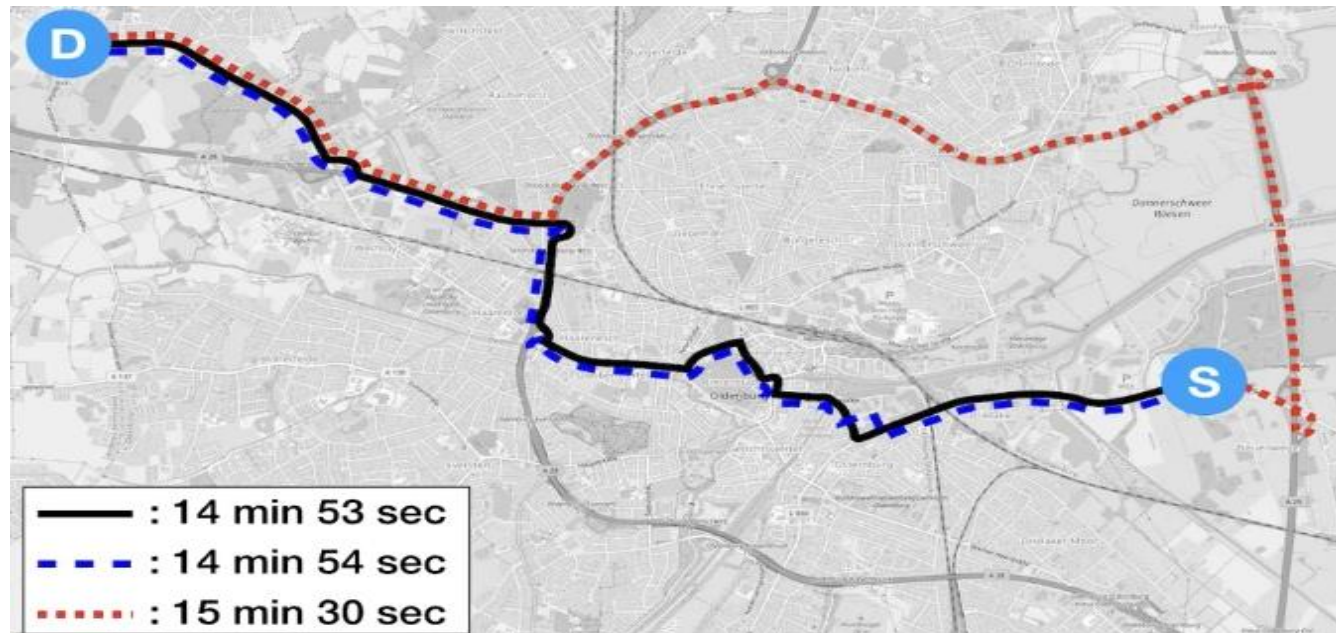


C:\Users\ZenBook\Desktop

```
(1,2,5)
(2,3,7)
(2,4,6)
(3,4,5)
(4,1,2)
```

ปัญหาเส้นทางที่สั้นที่สุด (Shortest path) ในกราฟ

- ปัญหาเส้นทางสั้นที่สุด (shortest path) หมายถึงเส้นทางระหว่างสองเวอร์เท็กซ์ในกราฟที่ให้ความยาวรวมของเอจด์ (edge) น้อยที่สุด
- เป็นปัญหาของกราฟชนิดมีน้ำหนัก (weighted graph) เนื่องจากค่าน้ำหนักของเอจด์อาจแทนระยะทางหรือเวลา ระหว่างเวอร์เท็กซ์ได้ ดังนั้นเป็นการหาผลรวมที่น้อยที่สุดของเอจด์จาก u ไป v



Shortest path problem



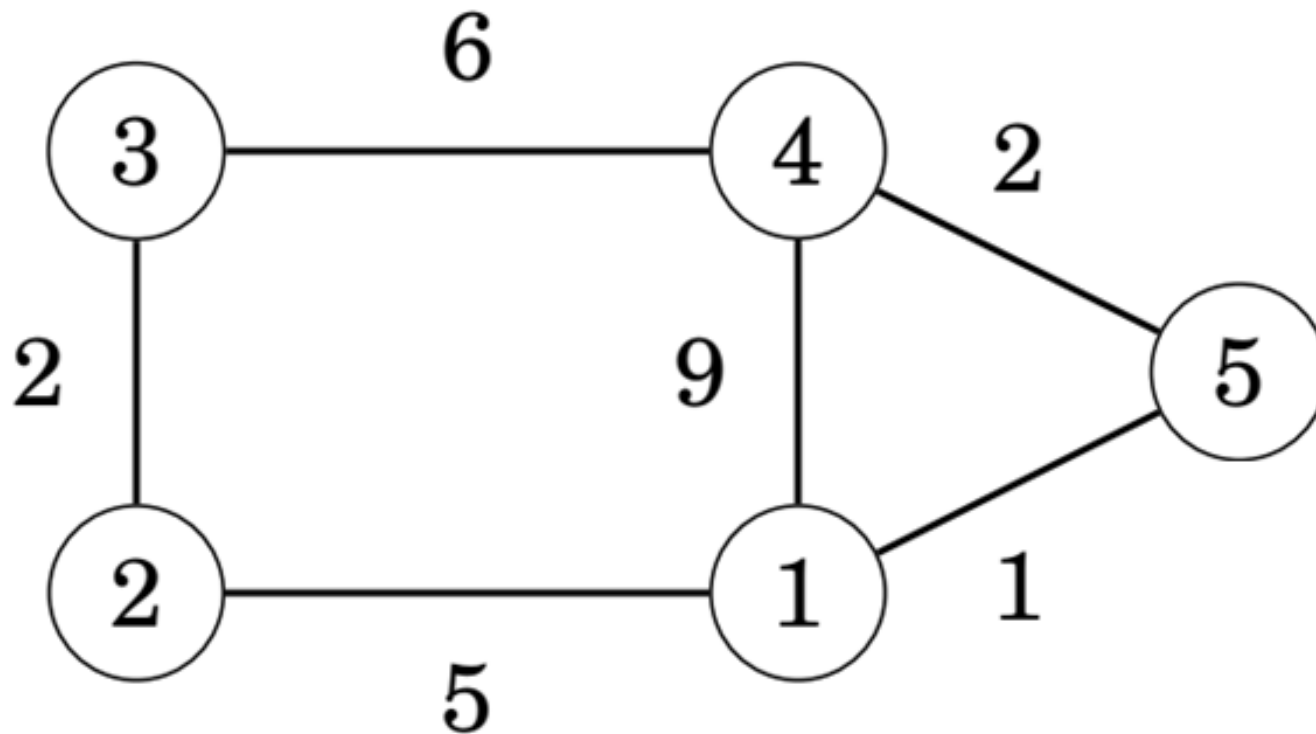
ปัญหาเส้นทางที่สั้นที่สุด (Shortest path) ในกราฟ

ปัญหาการหาเส้นทางที่สั้นที่สุด มักสามารถแบ่งออกเป็น 2 ประเภท ได้แก่

1. การหาเส้นทางสั้นที่สุดโดยมีการกำหนดจุดเริ่มต้น (single source) เรียกปัญหานี้ว่า Single Source Shortest Path (SSSP)
 - อัลกอริทึม Dijkstra
2. การหาเส้นทางที่สั้นที่สุดที่เวอร์เท็กซ์ใดก็ได้ เรียกปัญหานี้ว่า All-Pair Shortest Path (APSP)
 - อัลกอริทึม Floyd-warshall

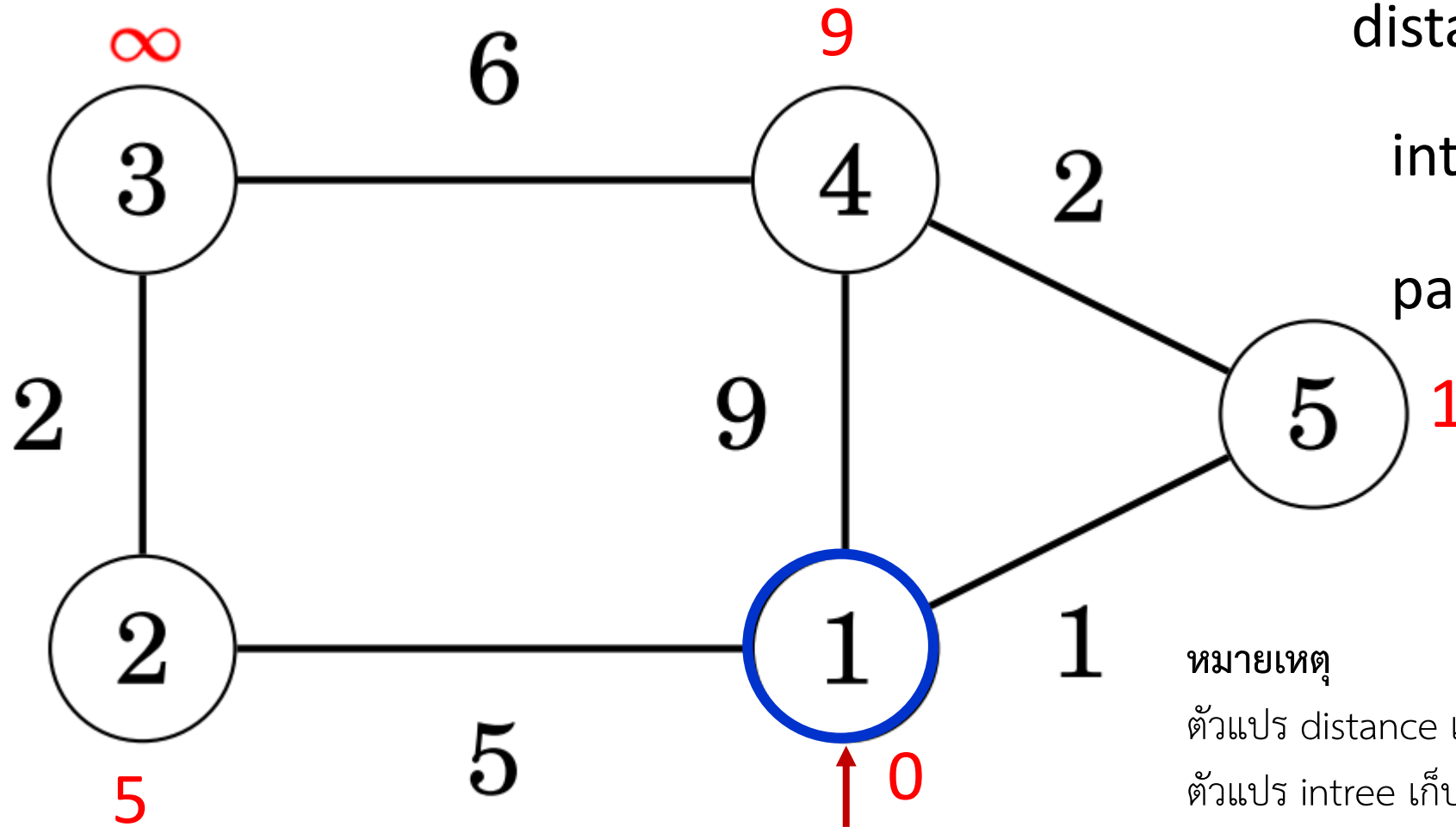
อัลกอริทึม Dijkstra

กำหนดจุดเริ่มต้น (single source) ในกราฟ จากนั้นจะเริ่มสร้างต้นไม้ (tree) โดยแผ่ไปยังเอดจ์ (edge) รอบข้าง โดยมีเงื่อนไขว่าเอดจ์ที่จะถูกเลือกเป็นคำตอบก็ต่อเมื่อระยะทางจาก source รวมกับน้ำหนักของเอดจ์นั้นมีค่าน้อยที่สุด (shortest path)



ต้องการหาเส้นทางสั้นสุดจากเวอร์เท็กซ์
1 ไปยังทุกเวอร์เท็กซ์ในกราฟ

อัลกอริทึม Dijkstra



distance

	1	2	3	4	5
distance	0	5	∞	9	1

intree

	1	2	3	4	5
intree	1	0	0	0	0

parent

	1	2	3	4	5
parent	-1	-1	-1	-1	-1

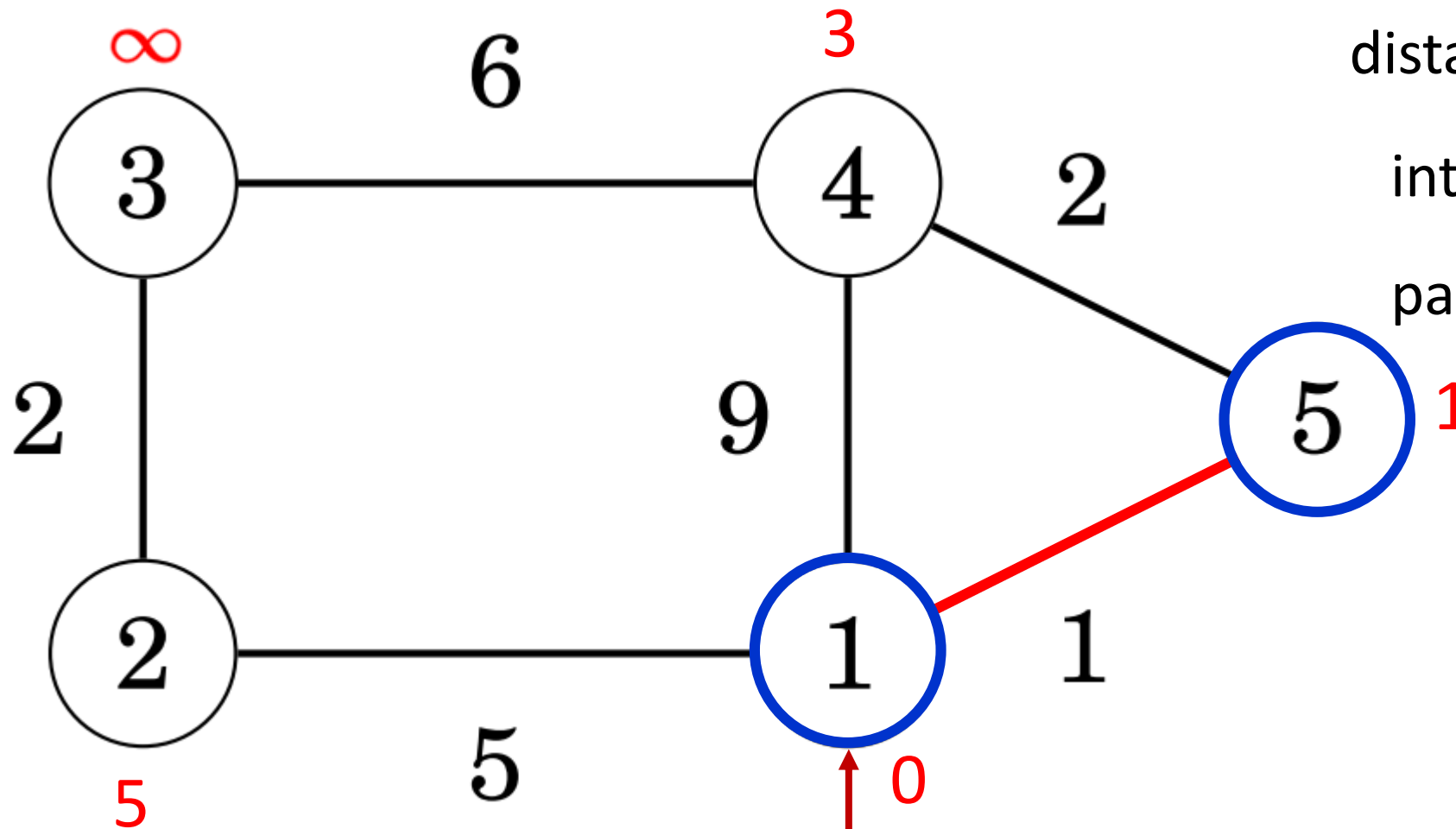
หมายเหตุ

ตัวแปร distance เก็บระยะทางจาก source ไปยังเวอร์เท็กซ์อื่น

ตัวแปร intree เก็บสถานะการเลือก vertex

ตัวแปร parent เก็บ path ของต้นไม้คำตอบ

Dijkstra algorithm



distance

1	2	3	4	5
0	5	∞	3	1

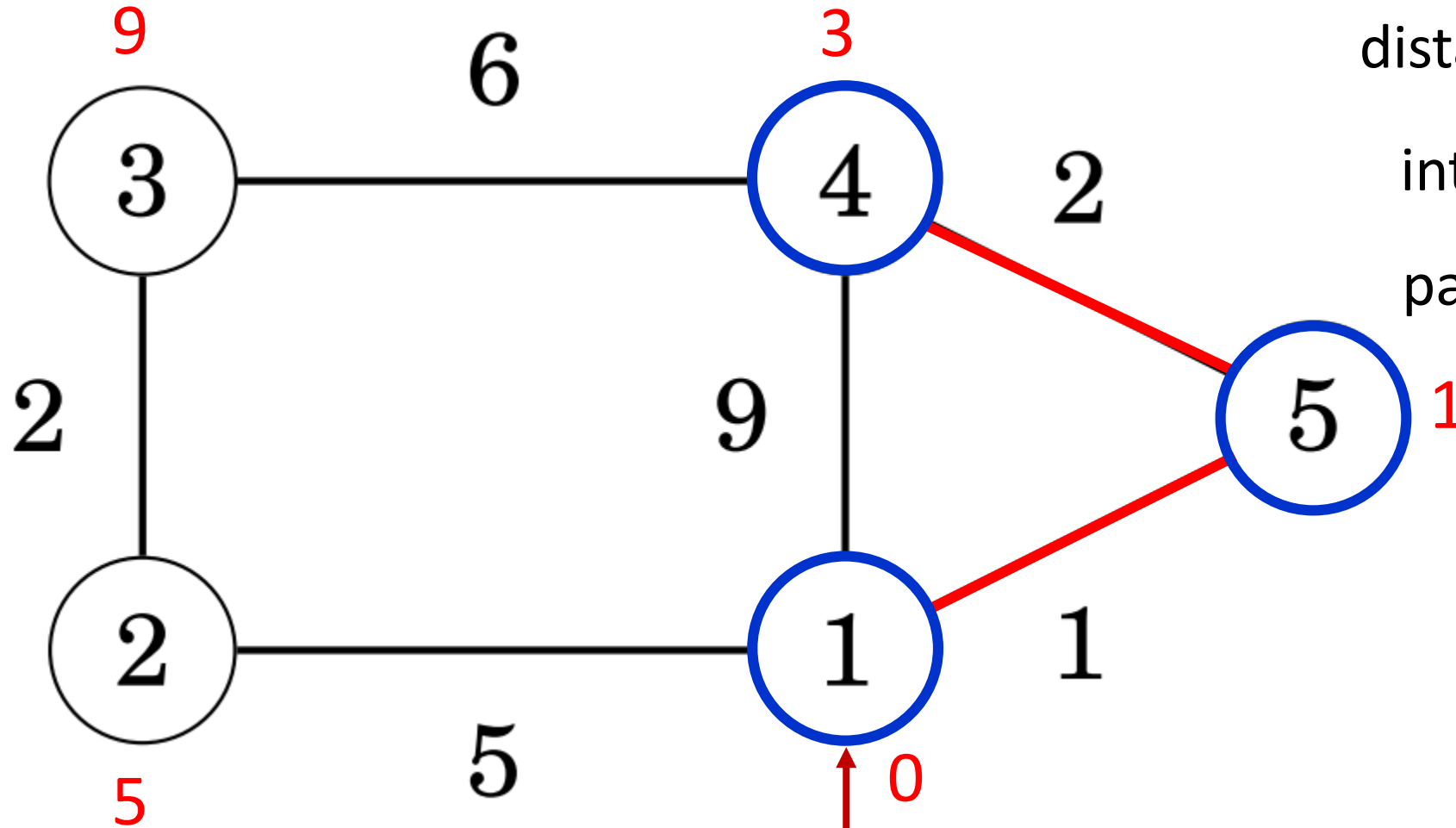
intree

1	2	3	4	5
1	0	0	0	1

parent

1	2	3	4	5
-1	-1	-1	-1	1

Dijkstra algorithm



distance

1	2	3	4	5
0	5	9	3	1

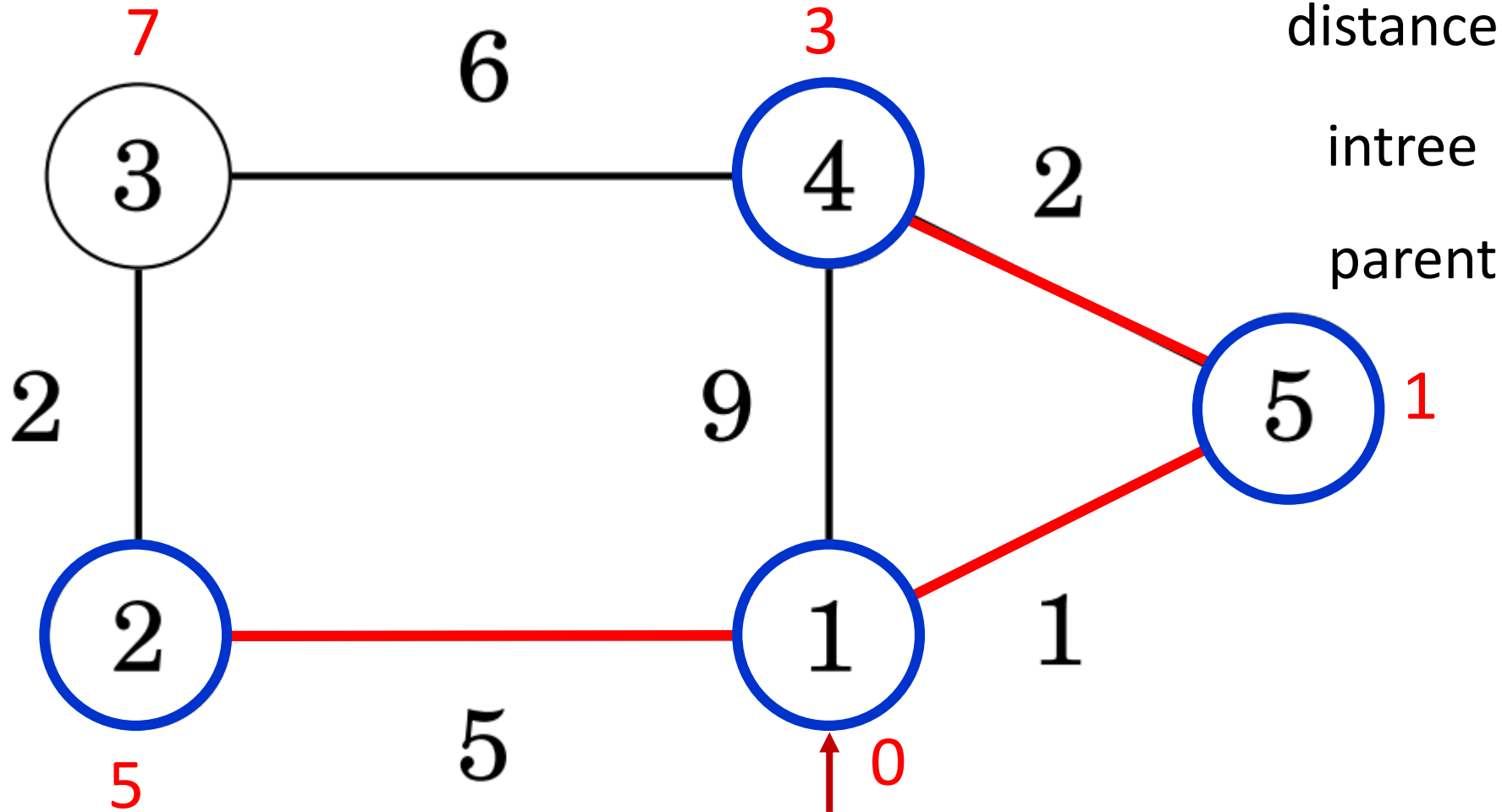
intree

1	2	3	4	5
1	0	0	1	1

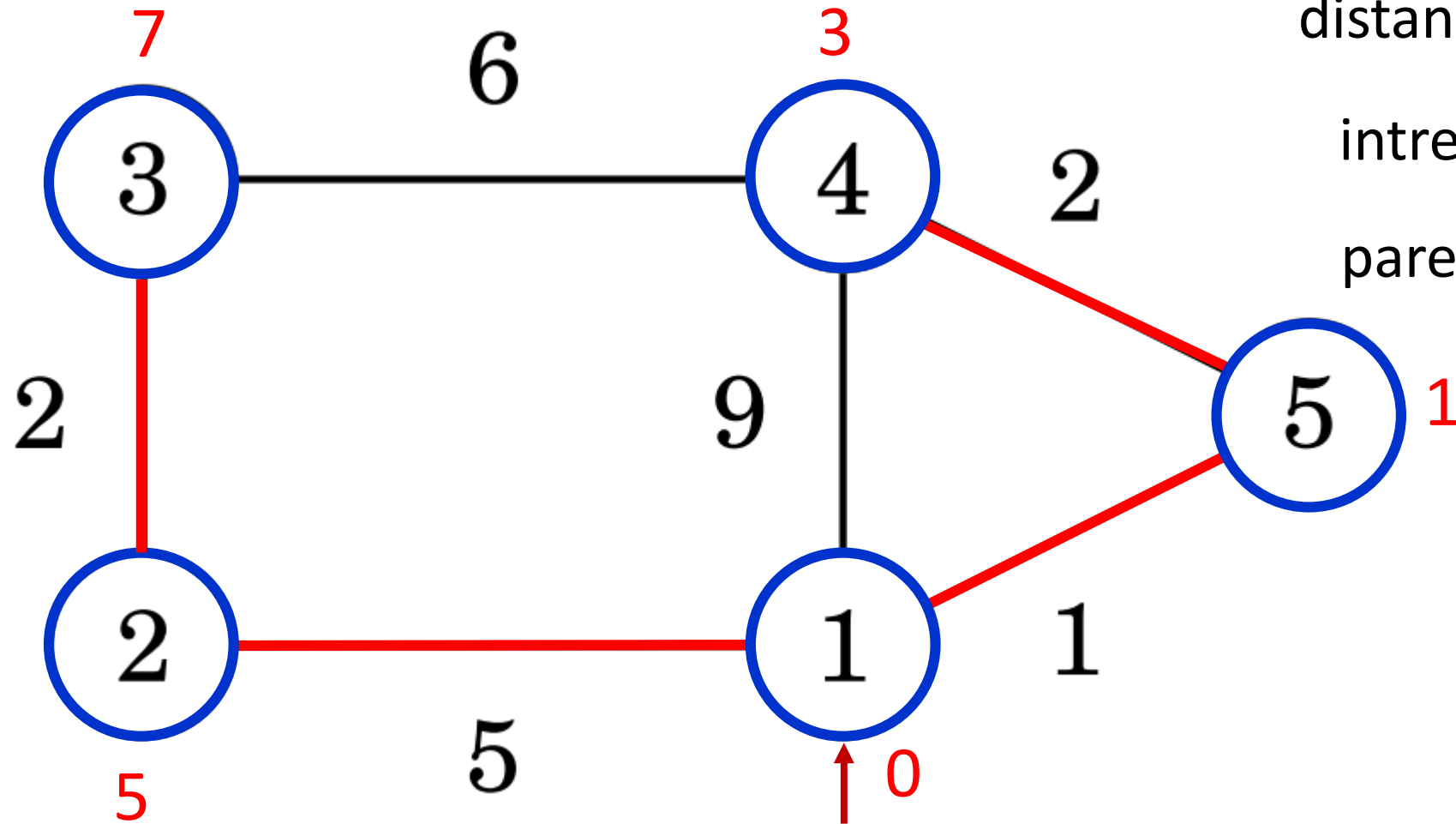
parent

1	2	3	4	5
-1	-1	-1	5	1

Dijkstra algorithm



Dijkstra algorithm



distance

1	2	3	4	5
0	5	7	3	1

intree

1	2	3	4	5
1	1	1	1	1

parent

1	2	3	4	5
-1	1	2	5	1



Dijkstra algorithm implemented in C

```
#define inf 50000
#define false 0
#define true 1

void dijkstra(int graph[][V], int src)
{
    int dist[V], inTree[V], parent[V];
    for (int i = 0; i < V; i++) // initial values
        dist[i] = inf, parent[i] = -1, inTree[i] = -1;
    dist[src] = 0; // start vertex
    for (int i = 0; i < V-1; i++)
    {
        int u = minDistance(dist, inTree); // find node u with minimum distance
        inTree[u] = true; // add u into the tree
        for (int v = 0; v < V; v++) // update all minimum weights
            if (inTree[v] == false && graph[u][v] > 0 && dist[u] + graph[u][v] < dist[v])
            {
                dist[v] = dist[u] + graph[u][v];
                parent[v] = u; // update parent
            }
    }
}
```

```
int minDistance(int dist[], int inTree[])
{
    int min = inf, min_index;

    for (int v = 0; v < V; v++)
        if (inTree[v] == false && dist[v] <= min)
        {
            min = dist[v];
            min_index = v;
        }
    return min_index;
}
```

Dijkstra algorithm in C++ STL

```
#include<bits/stdc++.h>
using namespace std;

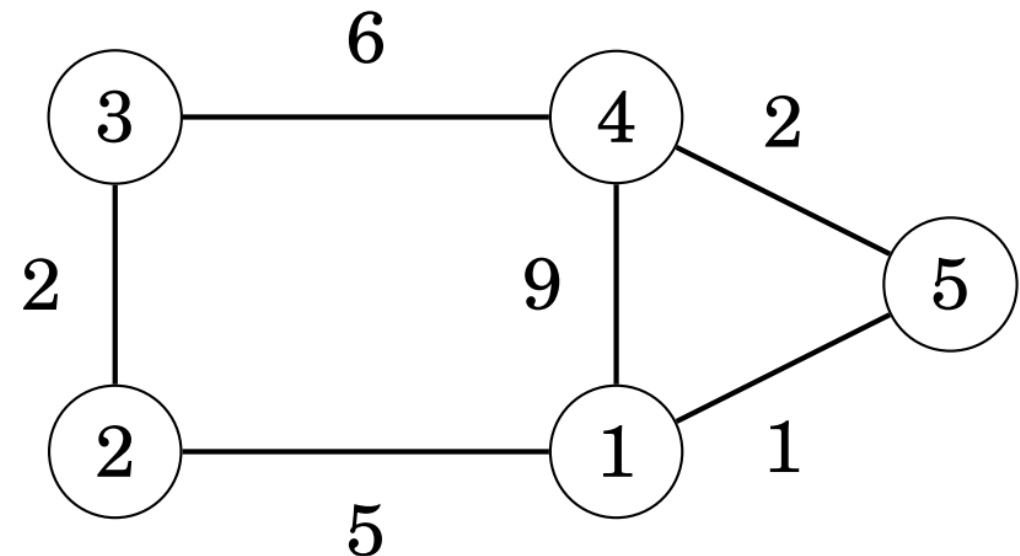
typedef pair<int, int> iPair;

int main()
{   int V = 5;
    vector<iPair > adj[V];

    addEdge(adj, 1, 2, 5);
    addEdge(adj, 1, 4, 9);
    addEdge(adj, 1, 5, 1);
    addEdge(adj, 4, 5, 2);
    addEdge(adj, 3, 4, 6);
    addEdge(adj, 2, 3, 2);

    Dijkstra(adj, V, 1);
}
```

```
void addEdge(vector <pair<int, int> > adj[],
int u, int v, int wt)
{
    adj[u].push_back(make_pair(v, wt));
    adj[v].push_back(make_pair(u, wt));
}
```





```
void Dijkstra(vector<pair<int,int> > adj[], int V, int src)
{
    priority_queue< iPair, vector <iPair> , greater<iPair> > pq;
    vector<int> dist(V, INF);           // distance from source

    pq.push({0, src});                  // push start vertex src
    dist[src] = 0;                      // initial distance of src

    while (!pq.empty())
    { int u = pq.top().second;          // get u from pq.
      pq.pop();

      for (auto x : adj[u])
      { int v = x.first;                // get v and weight from adj list.
        int weight = x.second;

        if (dist[v] > dist[u] + weight)
        { dist[v] = dist[u] + weight;    // Update weight of v
          pq.push( {dist[v], v} );      // push v into pq.
        }
      }
    }
}
```

Workshop 1 Dijkstra

กราฟ G ประกอบไปด้วย n เวกซ์ (vertex) และ m เอดจ์ (edge) แต่ละเอดจ์จะถูกแสดงในรูปของ u, v, w แทนเอดจ์จาก u ไป v น้ำหนัก w จงเขียนโปรแกรมเพื่อพัฒนาอัลกอริทึม Dijkstra เพื่อหาเส้นทางที่สั้นที่สุดจากเวกซ์เริ่มต้น s ไปยังเวกซ์ปลายทาง t

ข้อมูลนำเข้า

บรรทัดที่ 1 จำนวนเต็มบวก n, m, s, t แทนจำนวนเวกซ์ จำนวนเอดจ์ เวกซ์เริ่มต้น และเวกซ์ปลายทาง

M บรรทัดถัดไป แต่ละบรรทัดแทนเอดจ์ประกอบไปด้วย u, v, w หมายถึงเอดจ์เริ่มต้นจากเวกซ์ u ไป v และมีค่าน้ำหนัก w

ข้อมูลส่งออก

จำนวนเต็มแทนระยะทางที่สั้นที่สุดจาก s ไป t หากไม่มีเส้นทางจาก s ไป t ให้ใส่ -1

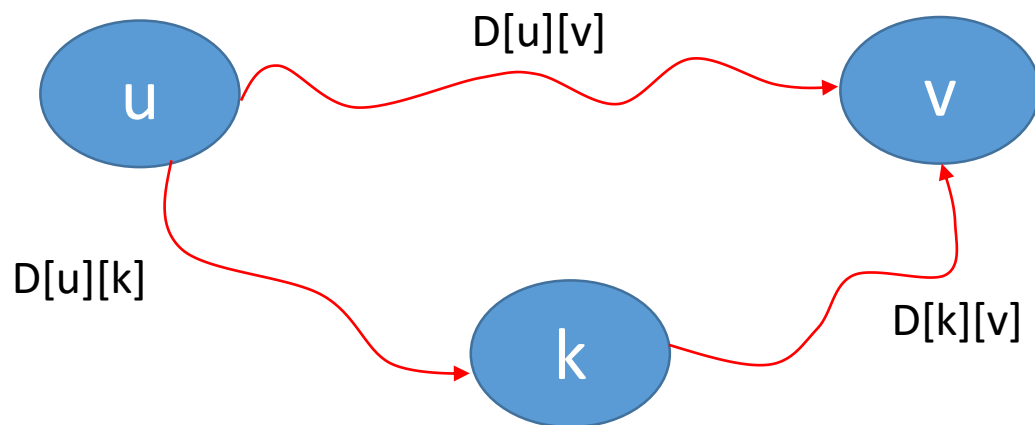
เงื่อนไข

$1 \leq n, m \leq 100$ $1 \leq s, t, u, v \leq n$ $0 < w \leq 10,000$

ตัวอย่างข้อมูลนำเข้า	ตัวอย่างข้อมูลส่งออก
5 7 1 5 1 2 5 1 3 3 2 3 1 2 4 6 3 4 4 3 5 7 4 5 2	9
3 2 1 3 1 2 1 2 1 2	-1

Floyd-Warshall algorithm

- อัลกอริทึมค้นหาเส้นทางที่สั้นที่สุดในกราฟชนิดถ่วงน้ำหนัก
- แตกต่างจากอัลกอริทึม Dijkstra โดยที่อัลกอริทึม Floyd-warshall จะสำรวจทุกเส้นทางที่สั้นที่สุดระหว่างทุกคู่ของเวอร์เท็กซ์ในกราฟ โดยรันอัลกอริทึมเพียงรอบเดียว
- อัลกอริทึม Floyd-warshall จะบันทึกและมีการปรับปรุงคำตอบ (ระยะทางสั้นที่สุดจาก u ไป v) โดยสร้างเมตริกซ์ระยะทาง (distance matrix) D



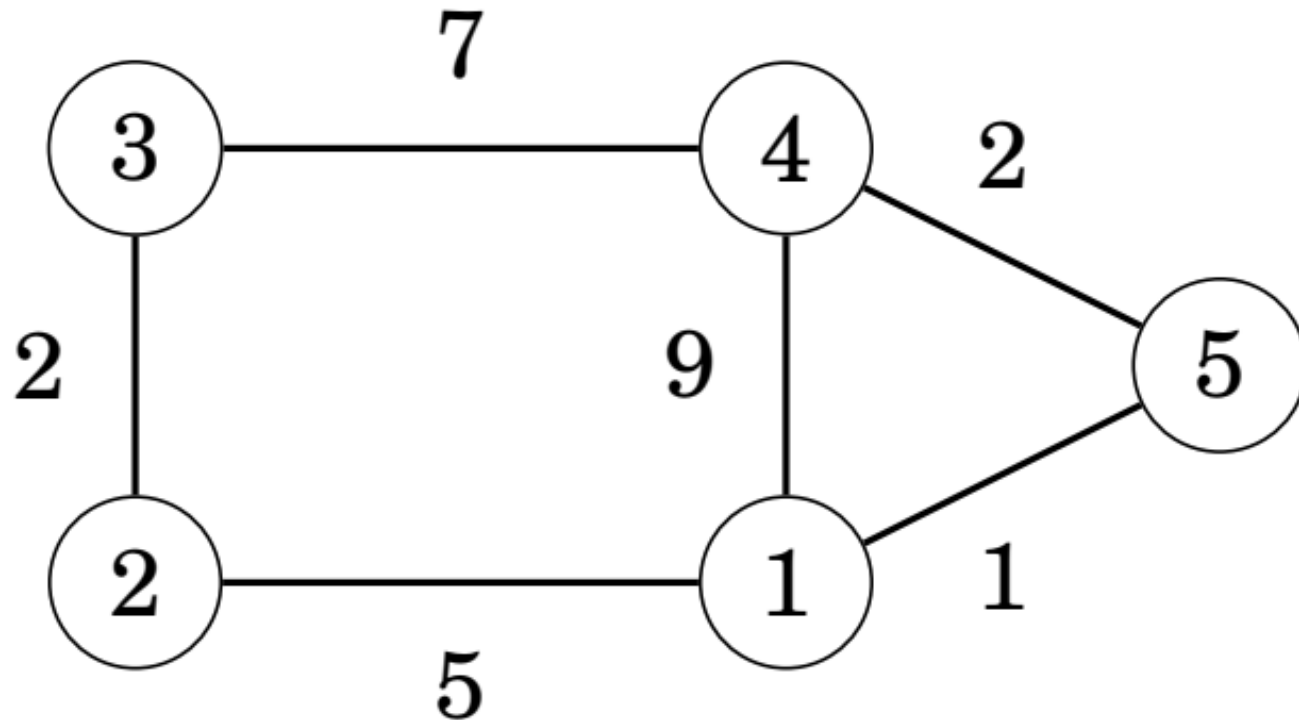
$$D[u][v] = \min \{ D[u][v], D[u][k] + D[k][v] \}$$

โดยที่ k เป็น intermediate vertex

Floyd-Warshall algorithm

กำหนดค่าเริ่มต้นให้กับเมตริกซ์ D ด้วยค่าน้ำหนักในแต่ละเอจต์ โดยมีเงื่อนไขเริ่มต้น

- ใส่ค่า inf. หากไม่มีเอจต์เชื่อมระหว่าง u และ v โดยตรง
- ระยะทางระหว่างเวกเท็กซ์ u ไป u จะมีค่าเท่ากับ 0

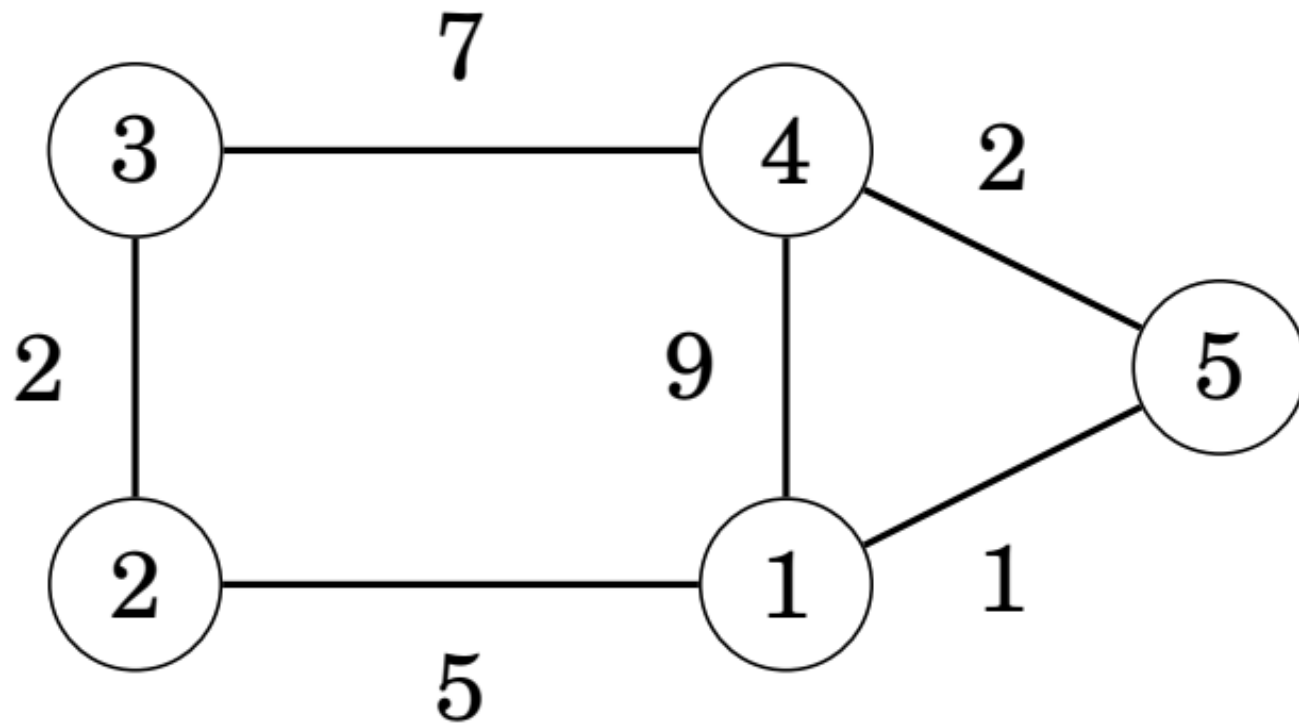


	1	2	3	4	5
1	0	5	∞	9	1
2	5	0	2	∞	∞
3	∞	2	0	7	∞
4	9	∞	7	0	2
5	1	∞	∞	2	0

อัลกอริทึม Floyd-Warshall

ให้ $D(0)$ เป็น adjacency matrix และทำการปรับปรุงเมตริกซ์ D โดยเลือกเวอร์เท็กซ์ 1 เป็น intermediate vertex

$$D[u][v] = \min \{ D[u][v], D[u][1] + D[1][v] \}$$



$D(0)$

	1	2	3	4	5
1	0	5	∞	9	1
2	5	0	2	∞	∞
3	∞	2	0	7	∞
4	9	∞	7	0	2
5	1	∞	∞	2	0



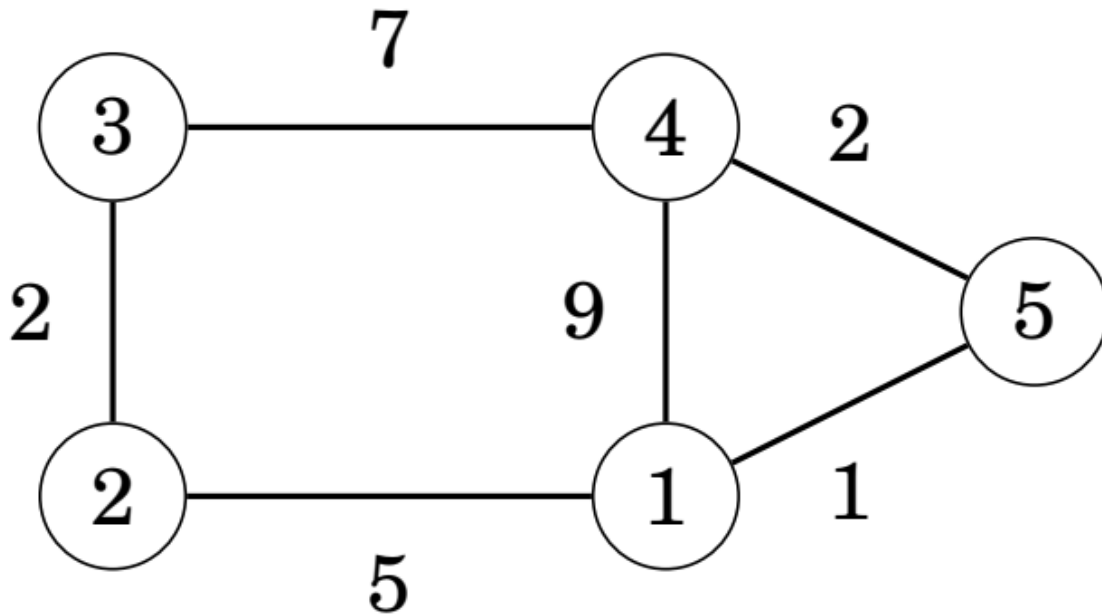
$D(1)$

	1	2	3	4	5
1	0	5	∞	9	1
2	5	0	2	14	6
3	∞	2	0	7	∞
4	9	14	7	0	2
5	1	6	∞	2	0

อัลกอริทึม Floyd-Warshall

ปรับปรุงเมตริกซ์ $D(1)$ โดยการเลือกเวอร์เท็กซ์ 2 เป็น intermediate vertex

$$D[u][v] = \min \{ D[u][v], D[u][2] + D[2][v] \}$$



$D^{(1)}$

	1	2	3	4	5
1	0	5	∞	9	1
2	5	0	2	14	6
3	∞	2	0	7	∞
4	9	14	7	0	2
5	1	6	∞	2	0

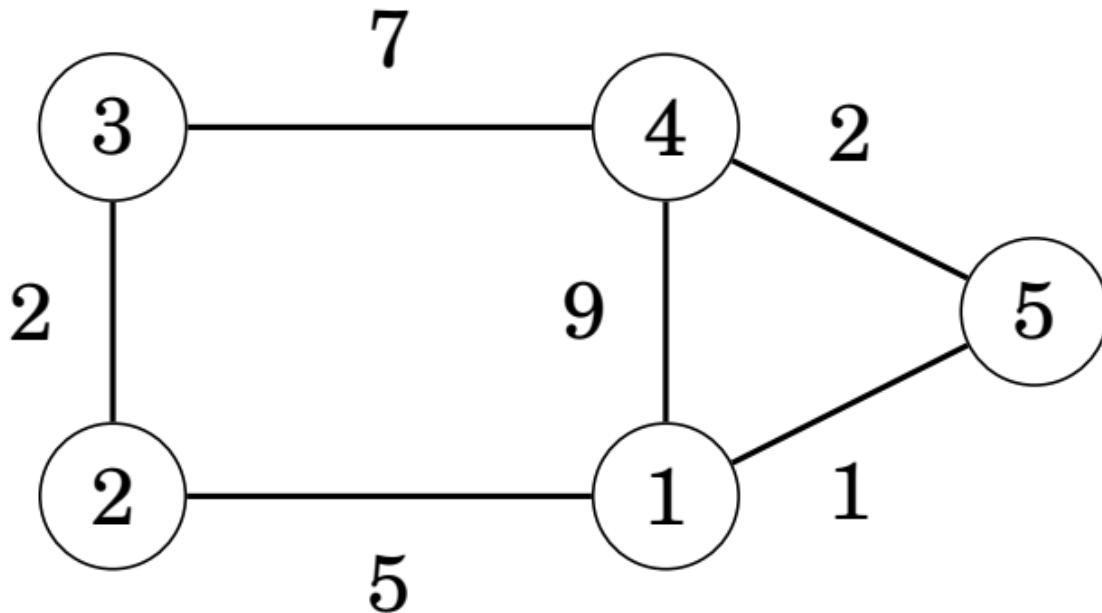
$D^{(2)}$

	1	2	3	4	5
1	0	5	7	9	1
2	5	0	2	14	6
3	7	2	0	7	8
4	9	14	7	0	2
5	1	6	8	2	0

อัลกอริทึม Floyd-Warshall

ปรับปรุงเมตริกซ์ $D(2)$ โดยการเลือกเวอร์เท็กซ์ 3 เป็น intermediate vertex

$$D[u][v] = \min \{ D[u][v], D[u][3] + D[3][v] \}$$



$D^{(2)}$

	1	2	3	4	5
1	0	5	7	9	1
2	5	0	2	14	6
3	7	2	0	7	8
4	9	14	7	0	2
5	1	6	8	2	0

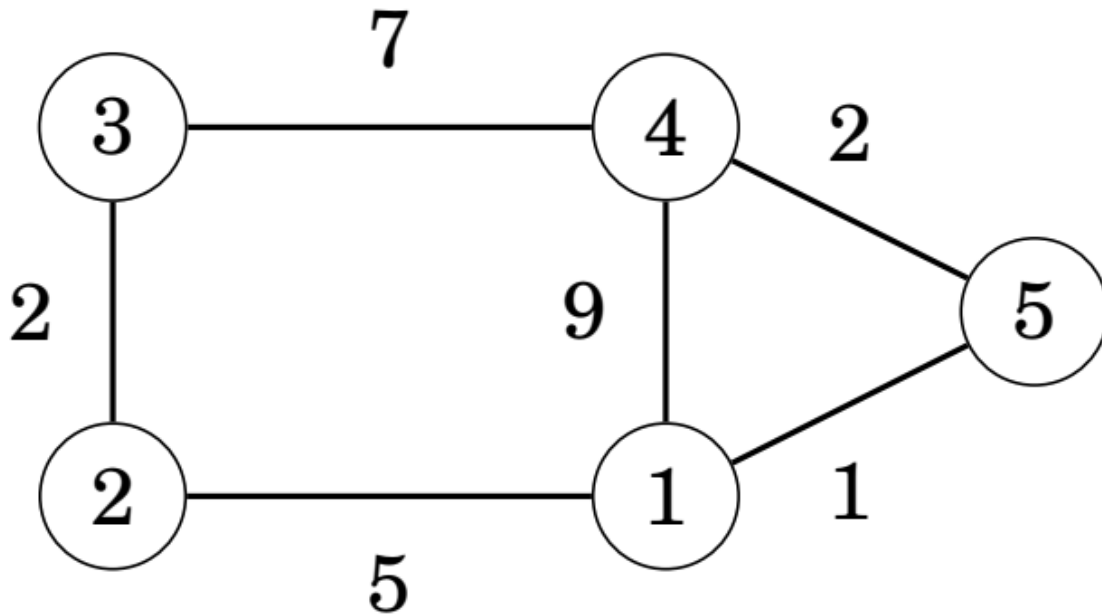
$D^{(3)}$

	1	2	3	4	5
1	0	5	7	9	1
2	5	0	2	9	6
3	7	2	0	7	8
4	9	9	7	0	2
5	1	6	8	2	0

อัลกอริทึม Floyd-Warshall

ปรับปรุงเมตริกซ์ $D(3)$ โดยการเลือกเวอร์เท็กซ์ 4 เป็น intermediate vertex

$$D[u][v] = \min \{ D[u][v], D[u][4] + D[4][v] \}$$



$D(3)$

	1	2	3	4	5
1	0	5	7	9	1
2	5	0	2	9	6
3	7	2	0	7	8
4	9	9	7	0	2
5	1	6	8	2	0

$D(4)$

	1	2	3	4	5
1	0	5	7	3	1
2	5	0	2	8	6
3	7	2	0	7	8
4	3	8	7	0	2
5	1	6	8	2	0

อัลกอริทึม Floyd-Warshall

```
void Floyd(int dis[][V])
{
    for(int k=0; k < V; k++)
    {
        for(int u=0; u < V; u++)
        {
            for(int v=0; v < V; v++)
            {
                if (dis[u][v] > dis[u][k] + dis[k][v]) {
                    dis[u][v] = dis[u][k] + dis[k][v];
                }
            }
        }
    }
}
```

```
#define inf 9999
void main()
{
    int graph[][V] = { {0, 3, inf, 7},
                        {8, 0, 2, inf},
                        {5, inf, 0, 1},
                        {2, inf, inf, 0}};

    Floyd(graph);
}
```

Workshop 2 Floyd-Warshall

กราฟทิศทางชนิดถ่วงน้ำหนัก G ประกอบไปด้วย n เวอร์เท็กซ์ (vertex) และ m เอดจ์ (edge) แต่ละเอดจ์จะถูกแสดงในรูปของ u, v, w แทนเอดจ์จาก u ไป v น้ำหนัก w จงเขียนโปรแกรมเพื่อพัฒนาอัลกอริทึม Floyd-Warshall เพื่อค้นหาเส้นทางที่สั้นที่สุดจากทุกเวอร์เท็กซ์ในกราฟ

ข้อมูลนำเข้า

บรรทัดที่ 1 จำนวนเต็มบวก n, m แทนจำนวนเวอร์เท็กซ์ จำนวนเอดจ์

M บรรทัดถัดไป แต่ละบรรทัดแทนเอดจ์ประกอบไปด้วย u, v, w หมายถึงเอดจ์เริ่มต้นจากเวอร์เท็กซ์ u ไป v และมีค่าน้ำหนัก w

ข้อมูลส่งออก

n บรรทัด แต่ละบรรทัดแสดงรายการจำนวนเต็ม n จำนวน แทนระยะทางสั้นที่สุดจากระหว่างเวอร์เท็กซ์ค้นด้วยช่องว่าง หากไม่มีเส้นทางระหว่างเวอร์เท็กซ์ แสดง inf

เงื่อนไข

$1 \leq n, m \leq 100$

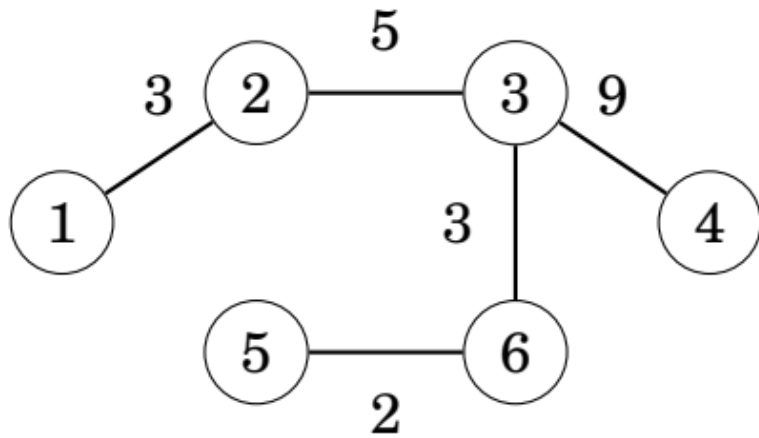
$1 \leq s, t, u, v \leq n$

$0 < w \leq 10,000$

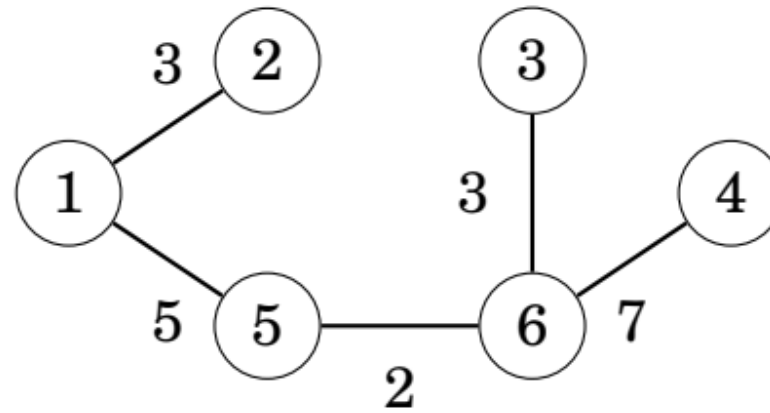
ตัวอย่างข้อมูลนำเข้า	ตัวอย่างข้อมูลส่งออก
4 6 0 1 1 0 2 4 1 2 2 1 3 5 2 3 1 3 0 3	0 1 3 4 6 0 2 3 4 5 0 1 3 4 6 0
5 5 1 2 1 1 3 2 2 3 1 2 4 6 3 4 1	0 inf inf inf inf inf 0 1 2 3 inf inf 0 1 2 inf inf inf 0 1 inf inf inf inf 0

ต้นไม้แผ่ทั่วที่เล็กที่สุด (Minimum spanning tree)

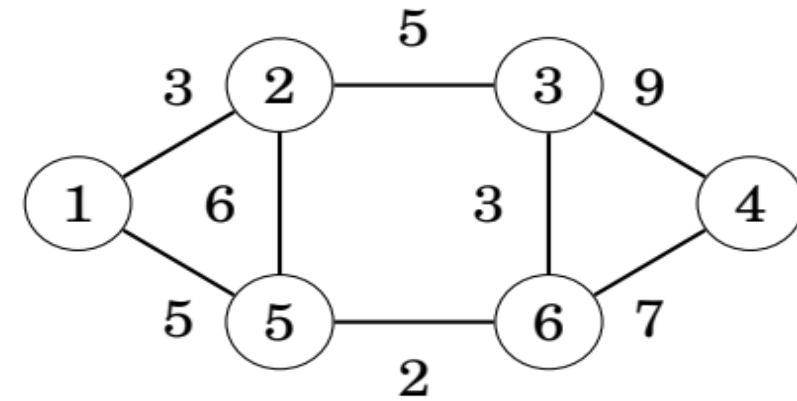
- ต้นไม้แผ่ทั่ว (Spanning tree) ของกราฟ หมายถึง acyclic graph ซึ่งเชื่อมโยงทุกเวอร์เท็กซ์ด้วยเอจด์บางเส้น ($|V| - 1$)
- ต้นไม้แผ่ทั่วที่เล็กที่สุด (Minimum spanning tree) คือต้นไม้แผ่ทั่วที่ผลรวมของเอจด์มีค่าน้อยที่สุด



$$3+5+9+3+2 = 22$$

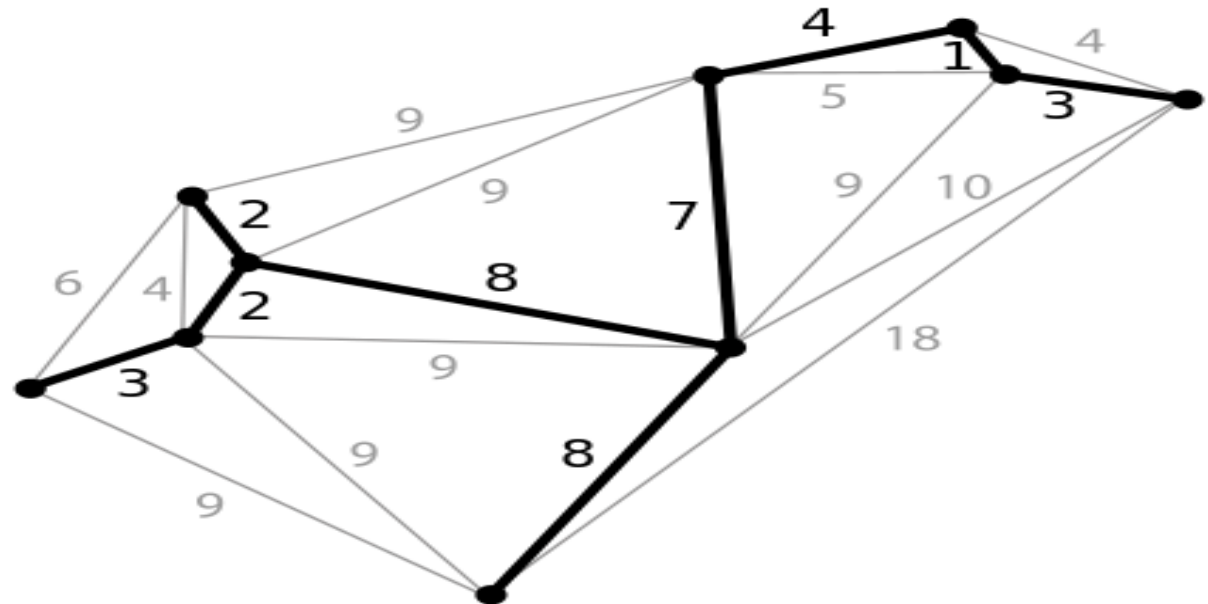


$$3+5+2+3+7 = 20$$



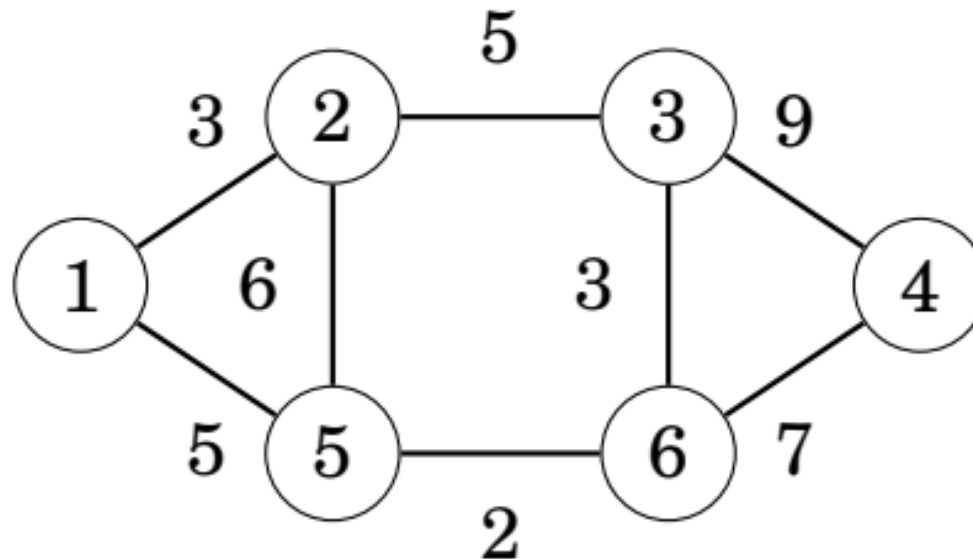
ต้นไม้แผ่ทั่วเล็กที่สุด (Minimum Spanning Tree)

- MST ในกราฟอาจมีได้มากกว่า 1 คำตอบ
- การสำรวจกราฟด้วย BFS /DFS เพื่อหา MST โดยตรงอาจไม่เหมาะสม เนื่องจากจำนวนต้นไม้แผ่ทั่วในกราฟมีมากเกินไปที่จะสร้างออกมาได้ทั้งหมด (exponential time)
- อัลกอริทึมสำหรับค้นหาต้นไม้แผ่ทั่วเล็กที่สุด
 - อัลกอริทึม Prim
 - อัลกอริทึม Kruskal

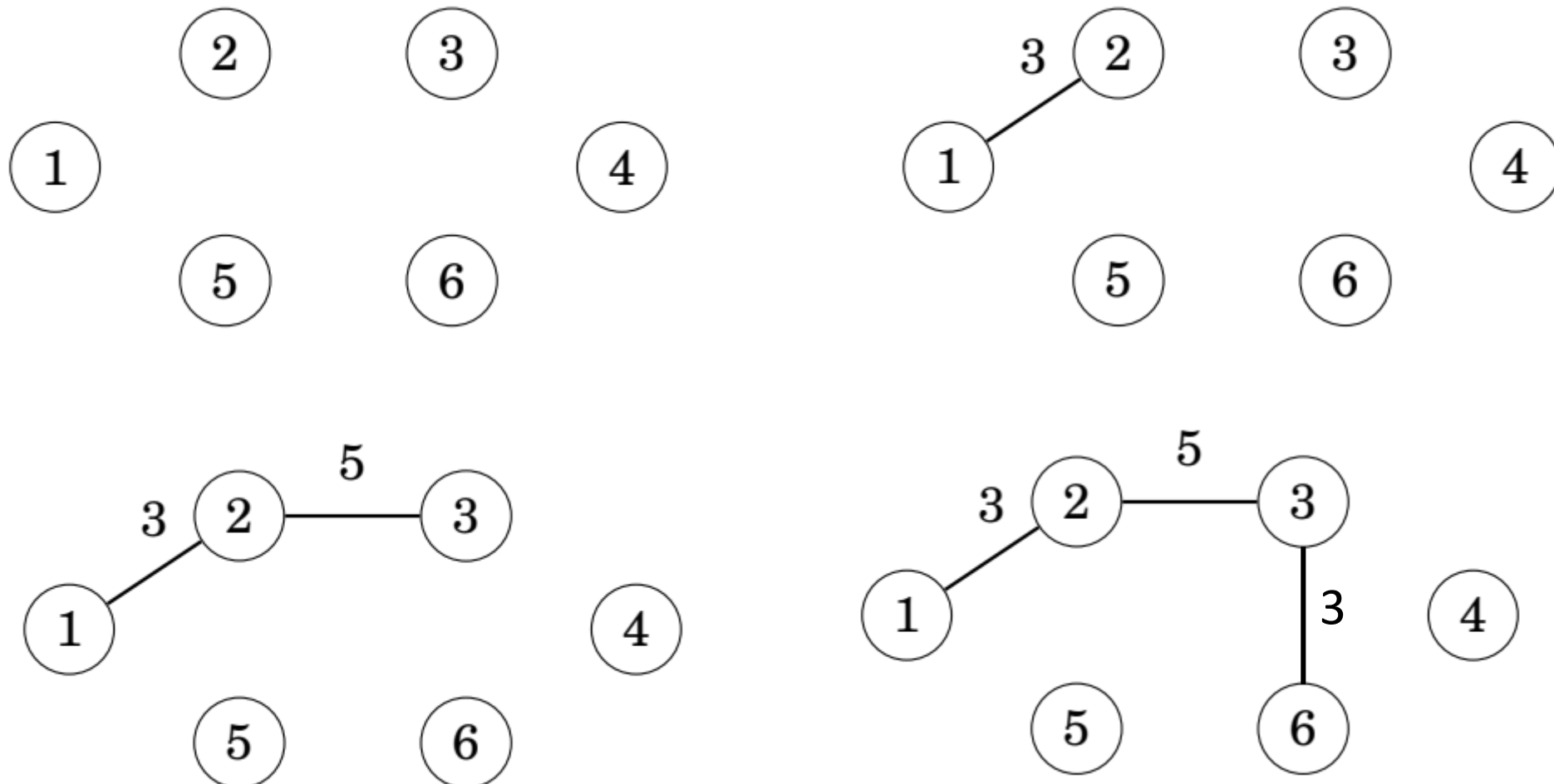
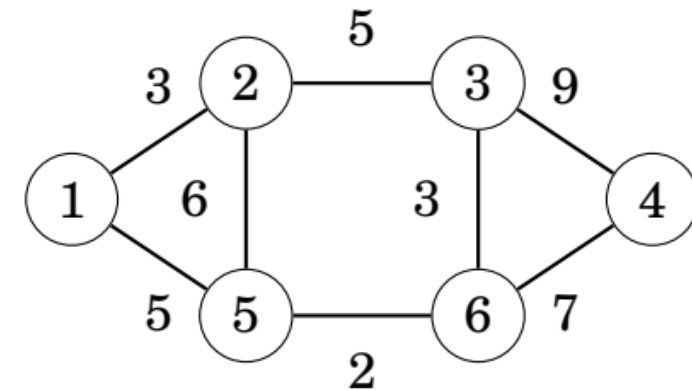


อัลกอริทึม Prim

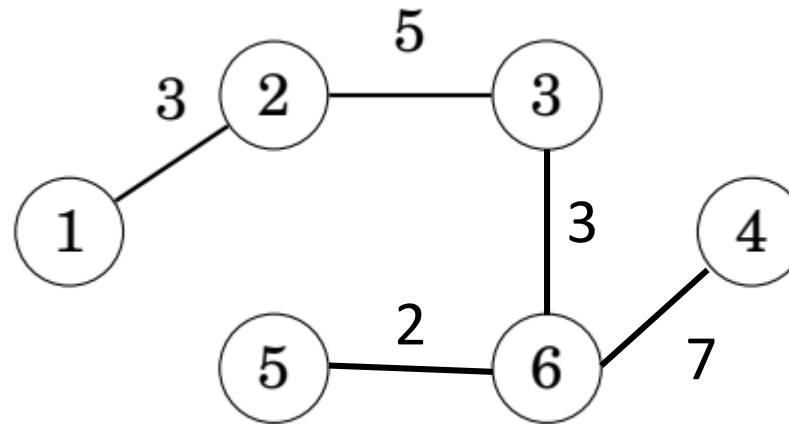
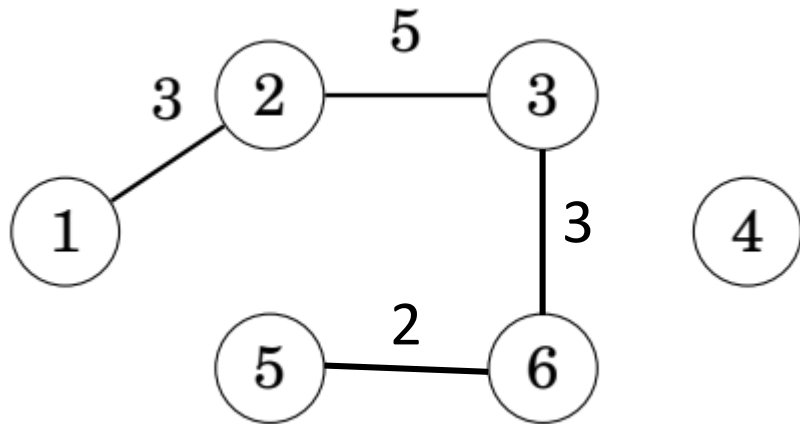
- เป็นอัลกอริทึมค้นหาต้นไม้แผ่ทั่วเล็กที่สุดที่คล้ายคลึงกับอัลกอริทึม Dijkstra
- โดยเริ่มต้นจะต้องกำหนดเวอร์เท็กซ์เริ่มต้นในกราฟ จากนั้นอัลกอริทึมจะเลือกเอจด์ที่มีค่าน้อยสุด (minimum-weight edge) จากเวอร์เท็กซ์เริ่มต้นและแผ่ทั่วจนครอบคลุมทุกเวอร์เท็กซ์
- อัลกอริทึม Prim จะเลือกเอจด์จนครบทุกเวอร์เท็กซ์ $(|V| - 1)$ โดยที่ V หมายถึงจำนวนเวอร์เท็กซ์



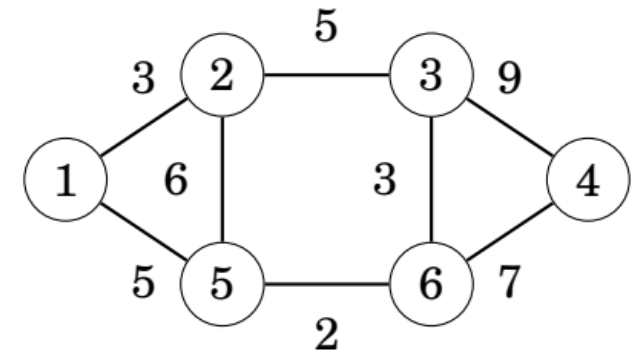
อัลกอริทึม Prim



อัลกอริทึม Prim

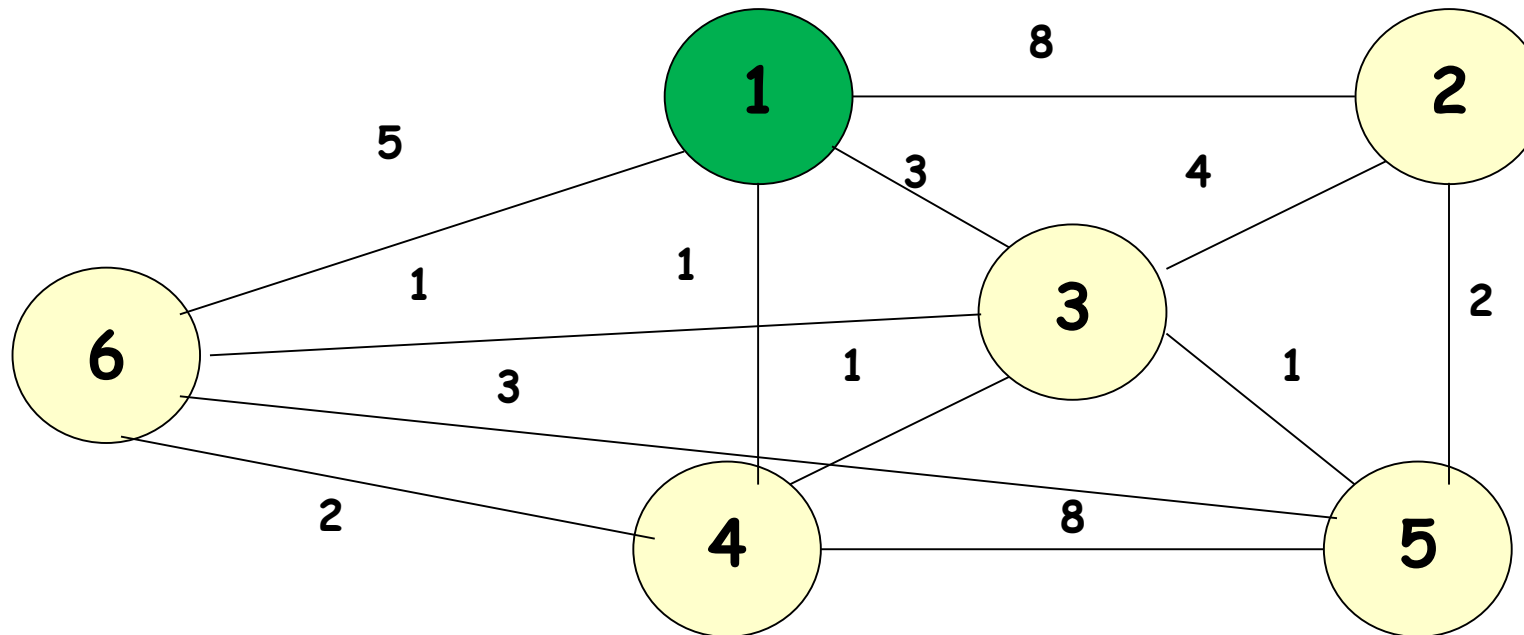


MST
 $3+5+2+3+7 = 20$



ทดสอบ

จงหา minimum spanning tree ของกราฟด้านล่างด้วยวิธี Prim





Workshop 3 Prim algorithm

กราฟชนิดถ่วงน้ำหนักไม่มีทิศทาง (undirected weight graph) G ประกอบไปด้วย n เวอร์เท็กซ์ (vertex) และ
เอดจ์ (edge) แต่ละเอดจ์จะถูกแสดงในรูปของ u, v, w แทนเอดจ์จาก u ไป v น้ำหนัก w จงเขียนโปรแกรมเพื่อค้นหา
ต้นไม้แผ่ทั่วที่เล็กที่สุด (minimum spanning tree: MST) ด้วยขั้นตอนวิธี Prim

ข้อมูลนำเข้า

บรรทัดที่ 1 จำนวนเต็มบวก n, m แทนจำนวนเวอร์เท็กซ์ จำนวนเอดจ์ในกราฟ คั่นด้วยช่องว่าง
 M บรรทัดถัดไป แต่ละบรรทัดแทนเอดจ์ประกอบไปด้วย u, v, w หมายถึงเอดจ์ระหว่างเวอร์เท็กซ์ u ไป
 v และมีค่าน้ำหนัก w
บรรทัดสุดท้าย จำนวนเต็มบวก s เป็นเวอร์เท็กซ์เริ่มต้น

ข้อมูลส่งออก

จำนวนเต็ม k แทนผลรวมของเอดจ์ในต้นไม้แผ่ทั่วที่เล็กที่สุด

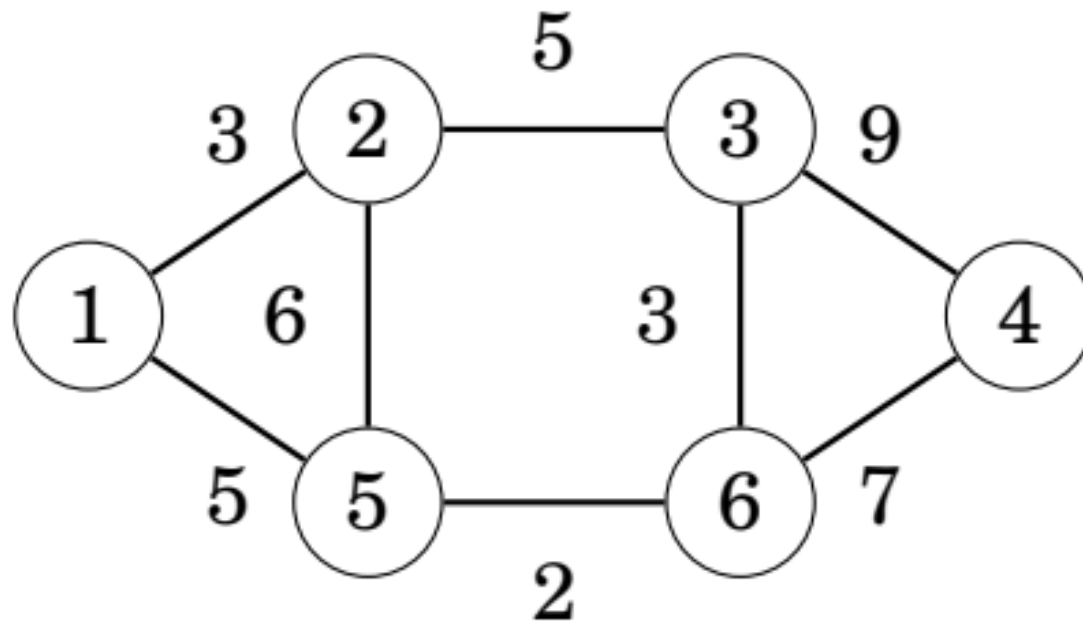
เงื่อนไข

- $1 \leq n, m \leq 100$
- $1 \leq s, t, u, v \leq n$
- $0 < w \leq 10,000$

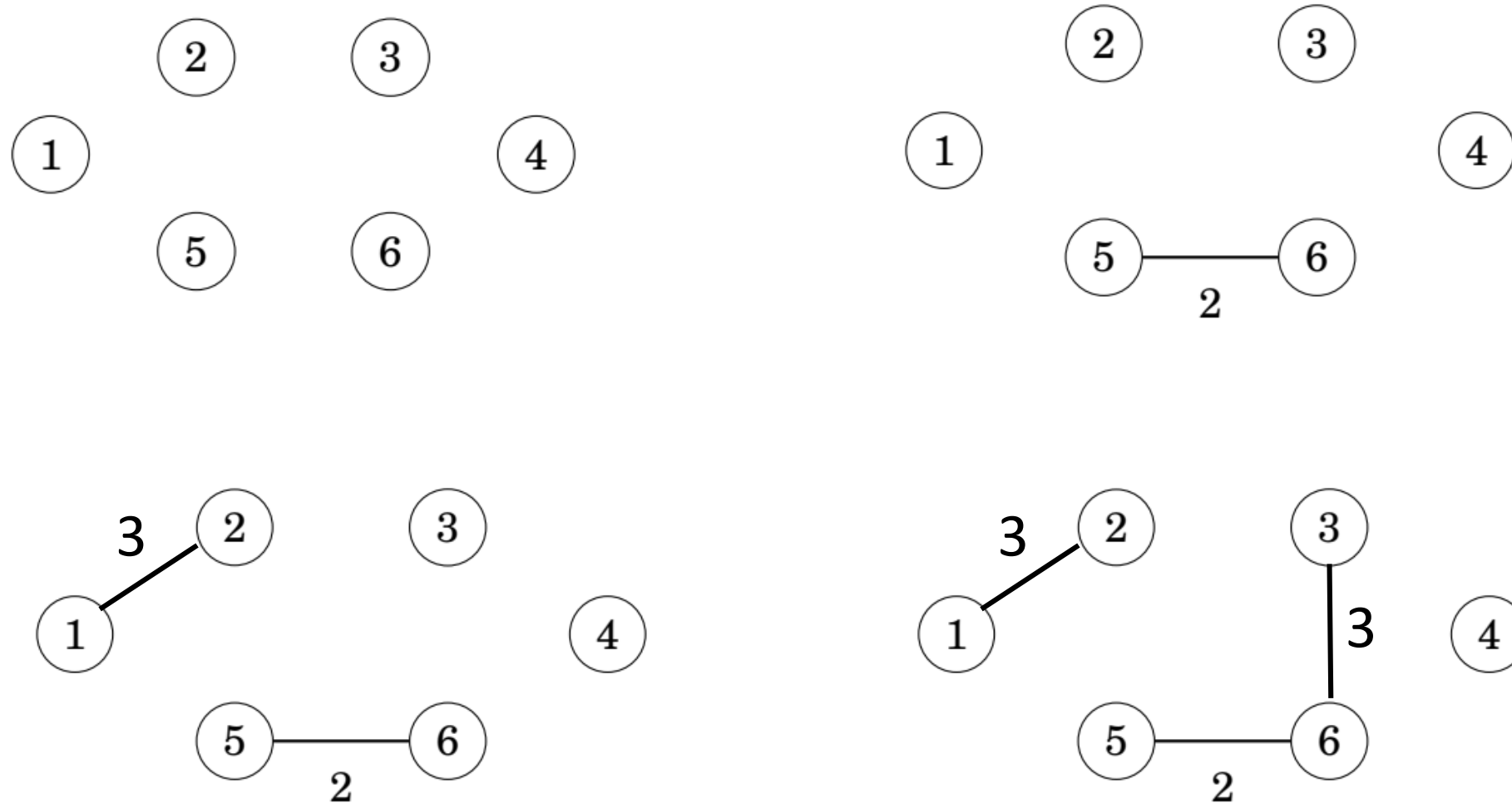
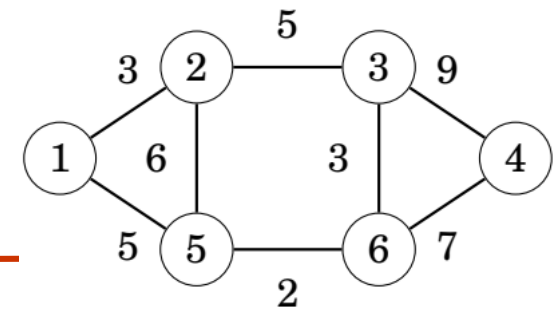
ตัวอย่างข้อมูลนำเข้า	ตัวอย่างข้อมูลส่งออก
4 5 1 2 2 1 3 1 1 4 4 2 3 3 3 4 5 1	7

Kruskal's algorithm

- เริ่มต้นจากเรียงลำดับเอดจ์ (edge) ตามค่าน้ำหนักจากน้อยไปมาก
- เพิ่มเอดจ์เข้าไปในคำตอบ หากเอดจ์นั้นไม่ทำให้เกิด วงวน (cycle) จนกระทั่งครอบคลุมทุกเวอร์เท็กซ์

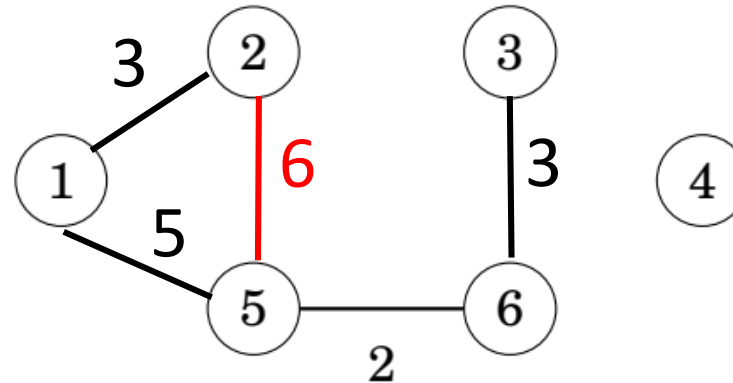
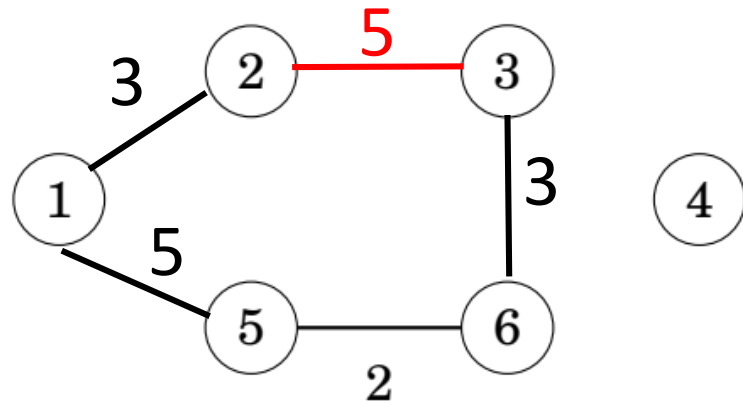
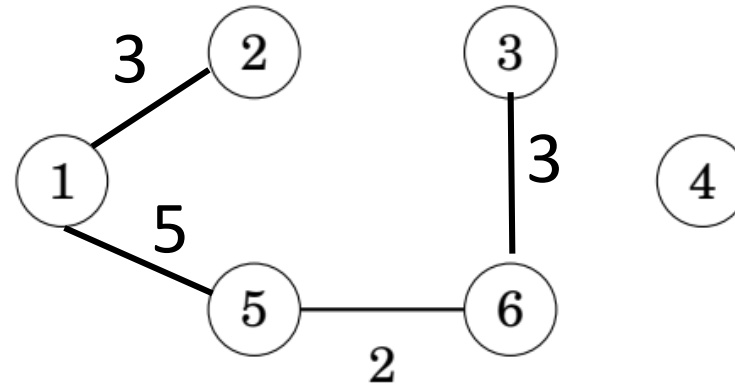
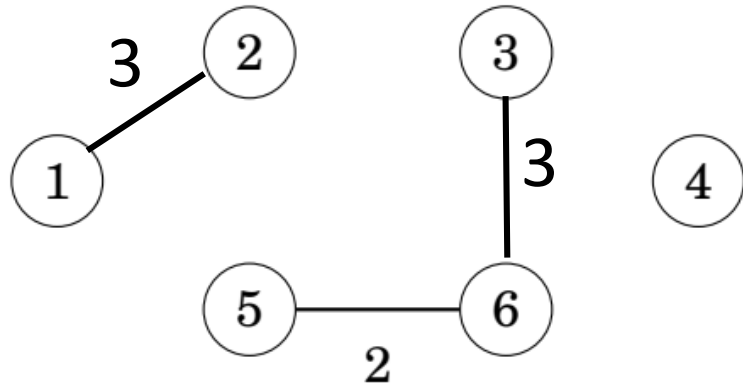
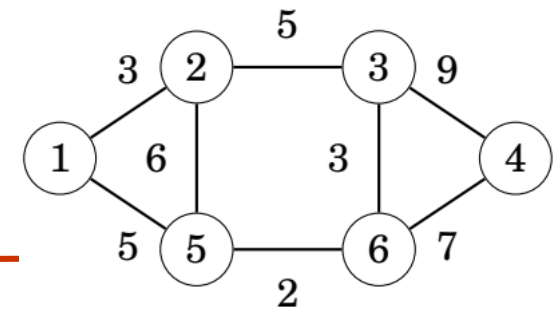


Kruskal's algorithm



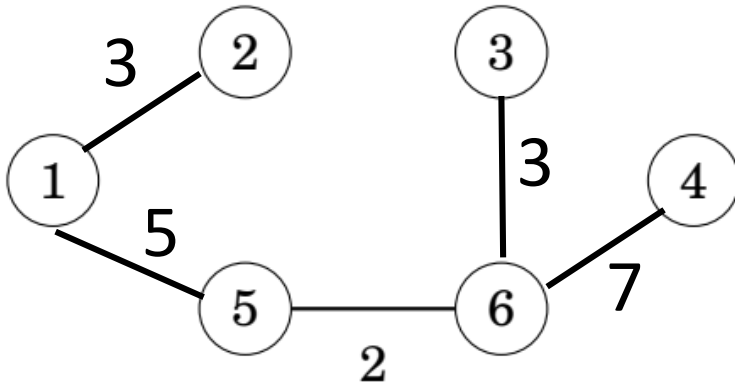
edge	weight
5-6	2
1-2	3
3-6	3
1-5	5
2-3	5
2-5	6
4-6	7
3-4	9

Kruskal's algorithm



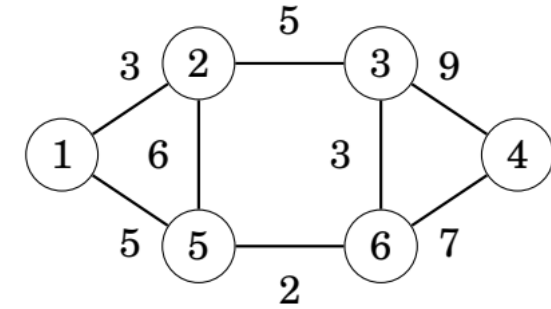
edge	weight
5-6	2
1-2	3
3-6	3
1-5	5
2-3	5
2-5	6
4-6	7
3-4	9

Kruskal's algorithm



MST

$$3+5+2+3+7 = 20$$



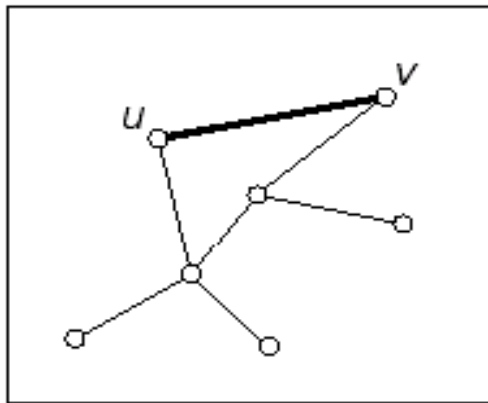
edge	weight
5-6	2
1-2	3
3-6	3
1-5	5
2-3	5
2-5	6
4-6	7
3-4	9

Kruskal's algorithm

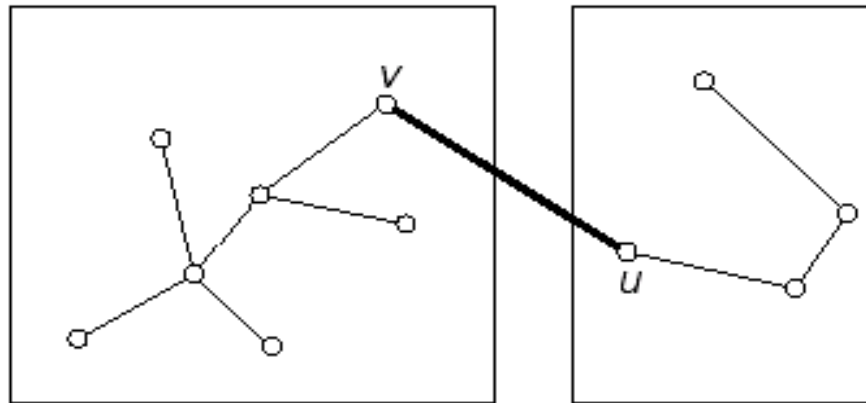
```
//Kruskal's algorithm for constructing a minimum spanning tree
//Input: A weighted connected graph  $G = \langle V, E \rangle$ 
//Output:  $E_T$ , the set of edges composing a minimum spanning tree of  $G$ 
sort  $E$  in nondecreasing order of the edge weights  $w(e_{i_1}) \leq \dots \leq w(e_{i_{|E|}})$ 
 $E_T \leftarrow \emptyset$ ;  $ecounter \leftarrow 0$       //initialize the set of tree edges and its size
 $k \leftarrow 0$                           //initialize the number of processed edges
while  $ecounter < |V| - 1$  do
     $k \leftarrow k + 1$ 
    if  $E_T \cup \{e_{i_k}\}$  is acyclic
         $E_T \leftarrow E_T \cup \{e_{i_k}\}$ ;  $ecounter \leftarrow ecounter + 1$ 
return  $E_T$ 
```

การตรวจสอบวงวน

- พิจารณาว่าแต่ละเวอร์เทกซ์ในกราฟเป็นต้นไม้ย่อย (sub-tree)
- ทุกครั้งที่เพิ่มเอดจ์ e_{uv} ต้องเช็คค่า u และ v อยู่ในต้นไม้ย่อยเดียวกันหรือไม่



(a)



(b)

- ต้องการอัลกอริทึมที่มีประสิทธิภาพที่ใช้ในการตรวจสอบว่าเวอร์เทกซ์ใดบ้างที่อยู่ในต้นไม้ย่อยเดียวกันบ้าง ซึ่งก็คือ Union-Find structure

อัลกอริทึม Union-Find

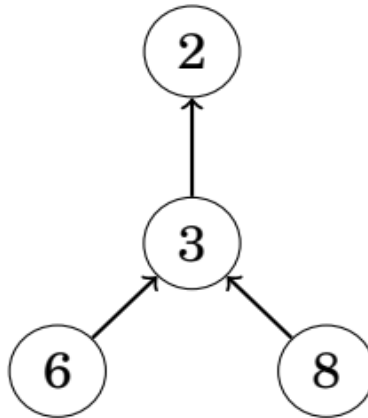
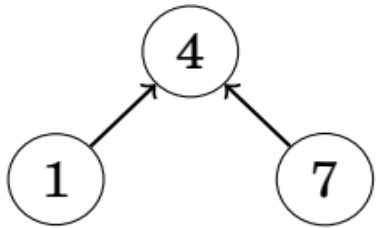
- Union-find เป็นอัลกอริทึมที่ถูกออกแบบมาเพื่อจัดการกับปัญหา disjoint sets เช่น $\{1,4,7\}$, $\{5\}$, $\{2,3,6,8\}$
- โดยแสดง disjoint sets ให้อยู่ในรูปของต้นไม้ย่อยและกำหนดให้สมาชิกตัวหนึ่งในแต่ละเซตเป็นตัวแทนของเซตนั้น (root)



	1	2	3	4	5	6	7	8
parent	4	-1	2	-1	-1	3	4	3

โอเปอเรชั่น Find

- ทำหน้าที่ค้นหาตัวแทนของสมาชิกในเซต เช่น ต้องการทราบว่า 6 มีตัวแทนเป็นใคร หรือ 7 มีตัวแทนเป็นใคร



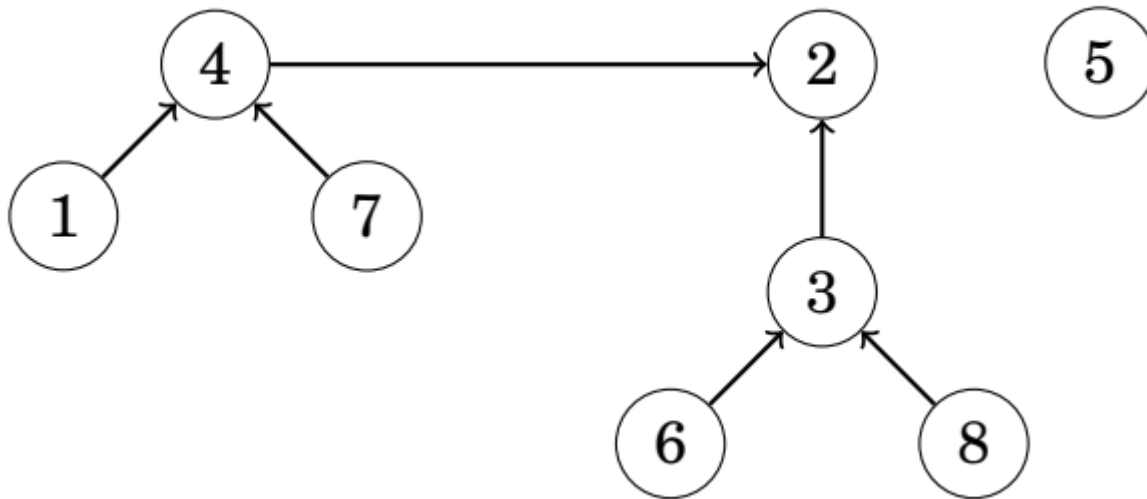
```

int find(int parent[], int i)
{ if(parent[i] == -1)
    return i;
  return find(parent, parent[i]);
}
  
```

	1	2	3	4	5	6	7	8
parent	4	-1	2	-1	-1	3	4	3

โอเปอเรชัน Union

- ทำหน้าที่รวมสองเซตย่อยเข้าด้วยกัน เช่น ต้องการรวม {1, 4, 7} และ {2, 3, 6, 8} เป็นเซตเดียวกัน
- วิธีการคือเปลี่ยนสถานะตัวแทนตัวใดตัวหนึ่งในอาร์เรย์ parent ให้เป็นลูกของตัวแทนอีกตัวหนึ่ง เช่น 4 จะกลายเป็นลูกของ 2 ซึ่งจะได้ผลลัพธ์เป็น {1, 4, 7, 2, 3, 6, 8}



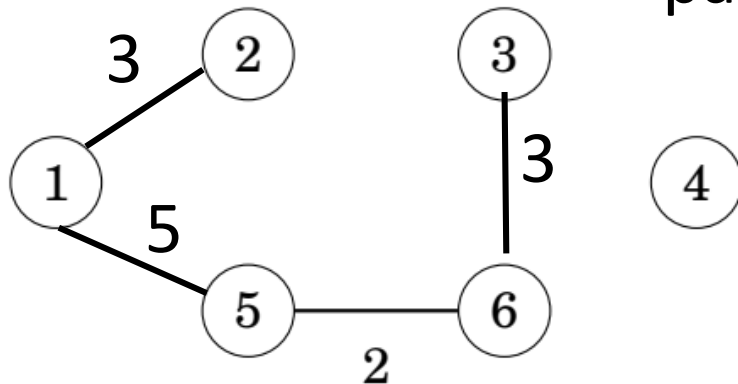
```
void union(int parent[], int i, int j)
{
    parent[j] = i;
}
```

	1	2	3	4	5	6	7	8
parent	4	-1	2	2	-1	3	4	3

การตรวจสอบวงวนด้วย Union-find

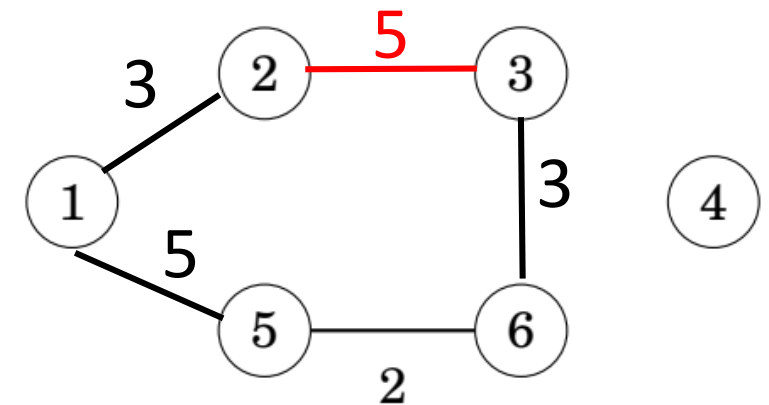
- สร้างฟังก์ชัน `is_cycle()` เพื่อตรวจสอบเอจ (u, v) ว่าอยู่ในเซตเดียวกันหรือไม่
- สมมติให้ เอจ (2, 3) ถูกเลือกเป็นลำดับถัดไป
- `is_cycle(2, 3) = ???`

```
int is_cycle(int u, int v)
{
    x = find(parent, u);
    y = find(parent, v);
    return (x==y)? true:false;
}
```



parent

1	2	3	4	5	6
2	-1	6	-1	1	5



Kruskal's algorithm

```
void kruskal(int g[][V])
{ int parent[V], ne=0, a, b, u, v;
  for(i=0; i<V; i++)
    parent[i] = -1;

  while(ne < V)
  { for (int i = 0, min = inf; i < V; i++)
    for (int j = 1; j<V; j++)
      if (g[i][j] < min)
      { min = g[i][j];
        a = u = i;
        b = v = j;
      }

    if ( !is_cycle(u, v) )
    { ne++;
      union(parent, u, v);
    }
    g[a][b] = g[b][a] = inf;
  }
}
```

```
int find(int parent[], int i)
{ if(parent[i] == -1)
  return i;
  return find(parent, parent[i]);
}
```

```
void union(int parent[],int i, int j)
{ parent[j] = i;
}
```

```
int is_cycle(int u, int v)
{ x = find(parent, u);
  y = find(parent, v);
  return (x==y)? true:false;
}
```



Workshop 4 Kruskal

กราฟชนิดถ่วงน้ำหนักไม่มีทิศทาง (undirected weight graph) G ประกอบไปด้วย n เวอร์เท็กซ์ (vertex) และ m เอจ (edge) แต่ละเอจจะถูกแสดงในรูปของ u, v, w แทนเอจจาก u ไป v น้ำหนัก w จงเขียนโปรแกรมเพื่อค้นหาค่าต้นไม้แผ่ทั่วที่เล็กที่สุด (minimum spanning tree: MST) ด้วยขั้นตอนวิธี Kruskal

ข้อมูลนำเข้า

บรรทัดที่ 1 จำนวนเต็มบวก n, m แทนจำนวนเวอร์เท็กซ์ จำนวนเอจในกราฟ คั่นด้วยช่องว่าง
M บรรทัดถัดไป แต่ละบรรทัดแทนเอจประกอบไปด้วย u, v, w หมายถึงเอจระหว่างเวอร์เท็กซ์ u ไป v และมีค่าน้ำหนัก w

ข้อมูลส่งออก

จำนวนเต็ม k แทนผลรวมของเอจในต้นไม้แผ่ทั่วที่เล็กที่สุด

เงื่อนไข

- $1 \leq n, m \leq 100$
- $1 \leq s, t, u, v \leq n$
- $0 < w \leq 10,000$

ตัวอย่างข้อมูลนำเข้า	ตัวอย่างข้อมูลส่งออก
4 5 1 2 2 1 3 1 1 4 4 2 3 3 3 4 5	7